Science of Computer Programming 166 (2018) 89–119

Contents lists available at ScienceDirect

Science of Computer Programming

www.elsevier.com/locate/scico

Towards naturalistic programming: Mapping language-independent requirements to constrained language specifications

Mariem Mefteh [a,*], Nadia Bouassida [a], Hanêne Ben-Abdallah [b]

[a] Sfax University, Mir@cl Laboratory, Sfax, Tunisia
[b] King Abdulaziz University, Jeddah, KSA and Sfax University, Mir@cl Laboratory, Sfax, Tunisia

ARTICLE INFO

Article history:
Received 22 November 2016
Received in revised form 7 May 2018
Accepted 21 May 2018
Available online 29 May 2018

Keywords:
Natural language processing
Patterns
Semantics
Multilingual requirements
Use case scenarios

ABSTRACT

This research paper presents a new approach that constitutes a first step towards programming using language-independent requirements. To leverage the needed programming effort, our approach takes requirements in the form of language-independent use case scenarios. Then, it generates the inputs of a code generator which, in turn, produces the corresponding code. To provide for the language-independence, our approach uses an enriched version of the semantic model, as a means to represent similar ideas possibly in different ways and in different natural languages. The enrichment consists of a set of patterns that it implements as XML code representing the information embedded in the use case scenarios. This intermediate representation can be processed to derive the inputs required by any code generator to produce code in a particular programming language. This paper illustrates the approach and its tool support for use case scenarios written in English and French, and semantic model patterns implemented as XML code that can be processed by the ReDSeeDS code generator. In addition, it presents the results of an experimental evaluation of the approach on use case scenarios (written in English and in French) belonging to five different systems. This evaluation quantitatively shows the ability of our approach: to extract ReDSeeDS inputs conforming to the expert's inputs with a high precision; to generate XML code elements conforming to the input with an encouraging performance as evaluated by the participating students (an F-measure ranging between 87.43% and 92.31%); and to generate Java code judged efficient by the participating programmers (an F-measure ranging between 66.4% and 93.43%).

© 2018 Elsevier B.V. All rights reserved.

1. Introduction

As argued by Knöll et al. [1], current programming techniques suffer from four main problems, namely the mental problem, the programming language problem, the natural language problem, and the technical problem. The *mental problem* reflects the obligation of adjusting the program ideas to the conditions of a specific programming language (like restructuring them in the form of classes, methods and attributes in the object-oriented languages). The *programming language problem* reflects the mandatory implementation of the same program ideas and algorithms in various ways depending on each programming language conditions. The *natural language problem* stems from the fact that people, working together from

* Corresponding author.
E-mail addresses: Mariem.mefteh.ch@gmail.com (M. Mefteh), Nadia.bouassida@isimsf.rnu.tn (N. Bouassida), HBenAbdallah@kau.edu.sa (H. Ben-Abdallah).

https://doi.org/10.1016/j.scico.2018.05.006
0167-6423/© 2018 Elsevier B.V. All rights reserved.

different countries, are obliged to document the developed software often in English, which makes the documentation task less productive and error prone. Finally, the *technical problem* occurs when developers spend most of their time thinking about and appropriately describing the main ideas embedded in the program [1]. The above problems result in time loss and lower productivity for software development companies.

Furthermore, due to the rapid progress of programming languages and their increasingly complex development environments, software developers find themselves in an endless race towards acquiring the necessary knowledge to be productive. While the underlying theory of programming languages is important to grasp in order to express the idea of a program, learning their syntax and structures should not be, however, a predicament to productivity. Indeed, instead of facing the actual challenging tasks of programming (describing, modeling and enhancing the actual idea of a program), programmers often end-up dealing with minor issues like choosing the right character set and doing number conversions.

To minimize the time of learning and wrestling with the programming languages' syntax/structures, on the one hand, and to overcome the above four problems of current programming techniques, on the other hand, "*naturalistic programming*" [1] [2] [3] advocates expressing *program ideas* in a natural language and to transform them into programming language structures. By writing computer programs using a natural language, naturalistic programming provides for the generation of programs in various programming languages, as-needed and with low costs. After all, programmers transform their thoughts into particular programming languages, while the ideas are almost the same even when expressed in different languages.

As a first step towards naturalistic programming, we propose, in this paper, an approach and its Code Recovery tool (CRec-tool) for extracting source code from language-independent requirements. More specifically, our approach extracts a code conforming to an existing code generator's syntax from requirements described as use case[1] scenarios, one of the most commonly used forms of requirements specification. Use case scenarios describe the future system functionality, from the user's perspective, with textual details that can be used to deduce/derive the internal system's (code). Our approach has the merit of accepting language-independent use case scenarios: It does not require sentences in a special form (e.g., SVO sentences within the scenarios steps [8]), and it accepts quite complex and ambiguous sentence structures. To handle these challenges, our approach uses the *semantic model* (originally [9] [10]) which we enhanced by structuring it through *patterns*; these describe the semantic model entities and implement them in terms of XML code. We decided to rely on the XML language due to its standardized structure that can be manipulated by several tools. The proposed patterns implementation can be exploited for different purposes, such as automatic text translation, domain analysis for software product lines [11] and source code generation – the focus of this paper. More specifically, the proposed patterns implementation are exploited to extract information from the use case scenarios in order to provide developers with useful code.

It is worth noting that several approaches were proposed to generate code from instructions written in a natural language (e.g., [12], [13], [14]). Some of these approaches follow model driven paths (e.g., [15] [16] [17] [18]). However, their majority is semi-automated and accepts inputs written in a syntax-controlled natural language, often only in the English language. In contrast, our approach removes these constraints thanks to the pattern-structured semantic model that it uses to generate a formal representation of the input language-independent scenarios. In addition, our approach benefits from the important advancement achieved in some projects like ReDSeeDS [8] for the code generation purpose: Our approach implements the semantic model patterns as inputs to any available code generator (ReDSeeDS in this paper). This makes our approach generic; that is, it can be used for any natural language (English and French are illustrated in this paper), and for any code generator (ReDSeeDS is illustrated in this paper). This merit is reached by implementing the semantic model patterns in XML code that is treated to extract the corresponding ReDSeeDS inputs which consist in RSL requirements and domain models. Furthermore, our approach generates Java code following the Model/View/Presenter (MVP) architectural pattern [19] which is a derivation of the Model/View/Controller (MVC) pattern. MVP keeps the same principles as MVC, except for the elimination of the interaction between the View and the Model layers, because it will be done through the Presenter layer. The MVP pattern is used mostly to build user interfaces (UI), and thus interactive systems. An interactive system is composed of GUIs (graphical user interfaces) and functional layers. The GUIs are rendered in the generated View layer and they enable the user to manipulate data. The functional layers contain the system treatment, performed in the Model layer in communication with the Presenter layer which organizes the data to be displayed in the View.

To show the advantages and limitations of our approach, this paper presents an experimental evaluation conducted by employing the CRec-tool on use case scenarios (written in English and in French) belonging to five different systems. This evaluation quantitatively examines the three stages of the approach. First, it shows the ability of our approach to extract the ReDSeeDS inputs conforming to the expert's inputs with a high precision (between 78.38% and 93%). Second, based on students' feedback, it shows that our approach generates XML code elements conforming to the input requirements with an F-measure ranging between 87.43% and 92.31%. Third, based on programmers' feedback, it shows that the efficiency of the Java code produced by our tool on four software applications is encouraging with an F-measure ranging between 66.4% and 93.43%.

The remainder of this paper is organized as follows: in Section 2, we overview approaches for information extraction from texts and source code generation from requirements. In Section 3, we present our approach and its tool for synthesizing

[1] In this paper, we adopt the original definition of "use case" as it was introduced by Jacobson [4]: "*A use case is a specific way of using the system by using some part of the functionality. It constitutes a complete course of interaction that takes place between an actor and the system.*" This notion has been explained by other researchers, such as Cockburn [5], Constantine et al. [6], Bittner [7], etc.

Java code from language-independent use case scenarios. Our approach is illustrated through the GoPhone[2] case study [20]. In Section 4, we discuss the experimental results of our approach and their threats to validity. Finally, in Section 5, we summarize the paper and highlight its future extensions.

2. Related work

This section overviews the state of the art pertinent to information extraction from texts, and source code generation from requirements.

2.1. Works on information extraction from texts

There is a large body of literature that tackles information extraction from texts. For instance, Glavas et al. [21] proposed the event graphs to extract event-based information from text. Their system performs anchor (i.e., a word that conveys the core meaning of an event like "*killed*" or "*bombing*") extraction, argument (i.e., protagonists and circumstances of events like *agent*, *time*, *location*) extraction, and relation extraction (i.e., temporal relation extraction and event co-reference resolution). Because this system deals only with events (i.e. the happening situations), it cannot be used to extract source code from natural language requirements where various naturalistic entities (types for programming, which are inspired by natural-language notions [1]) offer important information.

On the other hand, many studies focused on the *Frame Semantics* and the FrameNet project [22–24]. The former is based on lexicons made of a background knowledge whose structure is represented by "*frames*". The definition of a frame implies: (i) the discovery of participants, called *frame elements*, defined as their unique semantic roles to the situation, (ii) the mandatory participants of a frame called *core frame elements*, and (iii) the optional participants called *peripheral frame elements* [22]. The model of frame semantics has attracted the attention of a number of linguists interested in the lexicon of a specialty field [25]. No one can deny the importance of these works, in general, and the frame semantics approach, in particular. However, they are relevant to specific domains and frames, namely the already-defined ones. Besides, they attribute very specific semantic roles to the words of each frame. For example, for the sentence "*Kim DROVE his car*" [23], the semantic roles "*Driver*" and "*Vehicle*" are assigned respectively to the clauses "*Kim*" and "*his car*" by detecting the Frame "*Operate_vehicle*". However, to derive the source code elements from a natural-language instruction, it is not necessary to know the precise semantic role of a clause. Indeed, in this example, we do not need to know that "*Kim*" is a driver or that "*his car*" is a vehicle. In contrast, information on the dower of the action (i.e. "*Kim*") and the object that undergoes the action (i.e. "*his vehicle*") is sufficient. Besides, the type of a sentence (e.g. a *definition*, a *statement*, an *assignment*, etc.) cannot be determined by the frame semantics approach, although this fact is required for a relevant code extraction method. Moreover, the proposed techniques treated only one language, essentially English.

Obviously, the semantic model is adequately adapted in our approach because it gives appropriate and precise information which is relevant for the code derivation task thanks to the naturalistic entities that it relies on. Originally proposed in [9] [10], the semantic model structures input sentences in the form of mapping rules which must be adapted to specific needs. In this paper, we enhance this model with a clear representation of its entities in terms of patterns, in which we define particularly their structures and their implementation in XML. Indeed, the XML code of each entity can be easily used by any approach/tool thanks to its standardized structure.

2.2. Works on source code generation from requirements

There are some works that aimed to extract source code from requirements, following model driven path in most cases. For instance, Bousetta et al. [15], Kardoš et al. [16] and Fatolahi et al. [26] proposed methods that transform semi-automatically requirements of the Computation Independent Model (CIM) into a Platform Independent Model (PIM) represented as UML diagrams, as class, sequence, use case, activity, sequence and/or state machines diagrams. Despite their utility, these methods are semi-automated and cannot generate a source code. Similarly, Yue [27] introduced a method to generate automatically sequence and class diagrams from requirements (use case models) and established traceability links between them. Besides, Rhazali et al. [28] developed a technique for transforming service-oriented business models (the CIM) into web-based design models (the PIM) as use case diagrams, state diagrams, class diagrams, etc. However, these two methods do not allow generating a final source code. Moreover, Franců et al. [17] suggested a framework including a generator that produces an implementation as class methods. They also proposed a prototype that generates JEE applications using this technique. The major limit of this work is that it relies on the manual intervention of engineers to build a domain model required for the generator of the system implementation.

On the other hand, Smialek et al. [29] [18], Nowakowski et al. [8] and Kalnins et al. [30] proposed approaches that transform behavior scenarios written in a controlled natural language into UML models (like class diagrams, sequence diagrams, etc.) and the final code (following a Model/View/Presenter architecture). These approaches are based on a special

[2] This case study is based on a hypothetical context of the mobile phone company "Go-Phone" Inc and it has clone based Go-phone products, such as "S", "L", "Elegance", "Com", etc. [20].

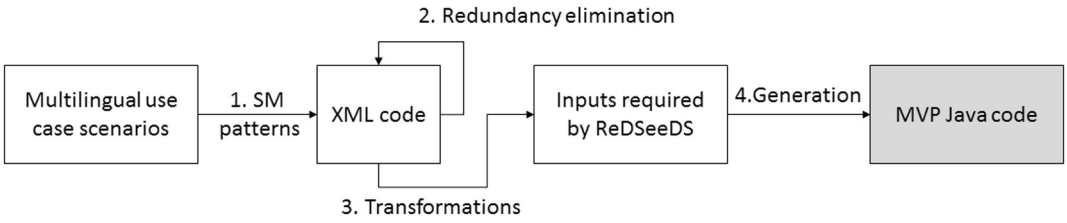


Fig. 1. Functional structure of our approach.

Requirements Specification Language (RSL) to express the use case scenarios. Their major limit is that the use case scenarios must be pre-processed and written in a semi-natural language. Besides, the sentences must be written in the SVO(O) grammar (i.e. in Subject+Verb+Object or Subject+Verb+Object+Object), which is an important restriction for writing the input requirements while there is no use of "*naturalistic*" entities like links, references, compression (see section 3.1), etc. For instance, a conditional sentence in RSL must be preceded by "=> *cond:*" in order to be treated. In the same context, Pathan et al. [31] developed a system that generates behavioral code for embedded applications from UML class and sequence diagrams. Similarly, Beggar et al. [32] proposed a code generator that allows producing complete source code for the Java platform from the sequence diagram of system internal behavior by applying a set of model transformations.

Other than using model driven paths, Cozzie et al. [13] proposed the Macho system which parses descriptions written in natural language into a simple program, by asking the programmer to provide one or more examples of correct input and output as unit tests. Moreover, Price et al. [33] proposed the prototype NaturalJava for creating, modifying and examining Java programs using natural language input. In this prototype, the user must describe explicitly the commands at the syntactical level of Java. In addition, Özcan et al. [34] introduced an intelligent natural language interface based on Turkish for creating Java class skeletons and listing the classes and their members. These techniques use simple mapping models which model nouns to objects and arguments, verbs to methods and adjectives to attributes. However, the semantics of a natural language sentence should not be exploited using only these types of mapping models. Indeed, there are other facts that should be taken into account when dealing with code generation. For this reason, we rely on the semantic model as a solution to hold (almost) all the semantics of the input requirements written in a purely natural language.

In summary, all of the aforementioned approaches present adopt different ways for code generation from requirements. Some of them follow a model-driven engineering path which generates UML diagrams instead of source code (e.g., [15][16][28]). The remaining approaches start from requirements written in a controlled natural language, in English, by following a specific syntax (e.g., [29][8][30]). Some other methods begin from natural language texts, written in one language (the English in most cases), and require human interventions to generate source code (e.g. [13][33][34]). Based on our analysis of the results obtained by applying these approaches, we notice that it would be interesting to develop an approach starting from quite complex and ambiguous requirements and extracting the corresponding source code, without the need for any expert intervention nor a specific formalization.

Compared to the above approaches, ours has the merit of accepting language-independent requirements. In other words, it does not need to transform manually these requirements into the syntax of a specific language. In addition, our approach is extendable to cover various implementation platforms, and highly automated. Moreover, it is independent from the domains of the target applications as it can be applied on any software application domain. Furthermore, one of the major advantages of our approach is its multi-lingual aspect. To bridge between languages, our approach uses a "*smart*" natural language processing technique (the *semantic model*) which relies on the semantics among the sentences, independently of their language.

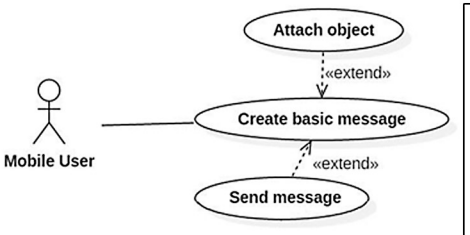
3. The approach

In this section, we describe our approach implemented using the CRec-tool (we refer the reader to the Appendix for more details about the implementation of this tool). Fig. 1 shows the functional structure of our approach. As illustrated in Fig. 1, the proposed method first applies the semantic model (SM) patterns on the use case scenarios in order to extract the corresponding implementations in XML (step 1 in Fig. 1). Then, our approach eliminates the redundancy among the code elements (step 2 in Fig. 1). Afterward, it transforms the resulting XML code into the inputs of the ReDSeeDS generator based on transformation rules (step 3 in Fig. 1). Finally, ReDSeeDS generates the target Java code following the MVP architectural pattern (step 4 in Fig. 1). In the remaining parts of this section, we detail the process followed by our approach. The different steps will be illustrated through the GoPhone case study proposed and published in 2004 within a technical report [20]. In this case study, the use cases were modeled by relying on the Cockburn's template [5], enriched with stereotypes that belongs to the context of software product lines (SPL) [35]. In this paper, we adapted the proposed scenarios to the Cockburn's template (e.g., by removing the notions in relation with SPLs). Moreover, we enhanced them by complex sentences, in order to prove the applicability of our approach on quite complex and ambiguous structures. Fig. 2 shows an extract taken from the use case scenarios of this case study.

Extract from the scenario of the use case: Create basic message

Use case name= Create basic message
Primary actors: User
[.....]
Main success scenario:
1. The user selects the menu-item "Send message".
2. The system dismisses the main menu screen.
3. The user selects the menu-item "Start new message".
4. The system displays a text editor screen.
5. The user enters the words of the message.
6. If T9 dictionary is activated, the system compares the words to the T9 dictionary.
7. The system validates and saves the message.
8. The use case ends.

Alternative flow:
6.1 If the T9 dictionary does not contain the words, then
6.1.1. The system plays a beep tone to indicate that it does not understand the symbol.
6.1.2. Go to 4.
7.1. In case of a full memory space of the mobile phone,
7.1.1. The system displays an error text, indicating that there is not enough memory space in the mobile phone.
7.1.2. The use case ends.
7.2. If the user selects the button "Attach object",
7.2.1. Perform "Attach object"
8.1. If the user selects the button "Send", then
8.1.1. Perform "Send message"
[.....]



Extract from the scenario of the use case: Attach object

Use case name= Attach object
Primary actors: User
[.....]
Main success scenario:
1. The model user selects the option "Add attachment"
2. The system builds the attachment choice list
3. The user selects the option "Contacts"
4. The system displays the contacts window and builds the contacts list.
5. The user selects a contact
6. The user selects the button "Done"
7. The system validates the contact
[.....]

Extract from the scenario of the use case: Send message

Use case name= Attach object
Primary actors: User
Secondary actor: Network
[.....]
Main success scenario:
1. The user selects the button "Send" which enables sending the message created by the user to the receiver.
2. The system displays the addressbook's contacts window.
3. The user chooses the contact who corresponds to the receiver of the message.
4. The system connects the network and sends the message to the recipient.
5. The network sends to the system an acknowledgement which consists of an ASCII character, sent by the recipient of a message (i.e. the destination) back to the user (i.e. the source) in case of the receipt of a recognizable block of data of specific size.
6. The system shows the acknowledgement to the user to inform him that the message was successfully sent.
7. The use case ends. [.....]

Fig. 2. An extract from the use case scenarios of our case study.

3.1. Extraction of the XML code from the use case scenarios

As mentioned in the introduction, this paper deals with the code extraction from the use case scenarios. In fact, several scenario templates were introduced in the literature. They differ in their narrative styles (e.g. continuous narrative [4], numbered sequence [36], actor/system conversation [37], stereotyped actions [38], etc.). The herein proposed approach is independent from the scenarios style. Indeed, we applied the semantic model on language-independent scenarios which follow the widely-used Cockburn's template [5]. The latter contains namely the fields *Goal*, *Primary* and *Secondary actors*, *Precondition*, *Success end condition* (i.e. the postcondition), *Description* (i.e. the main success scenario) and *Exceptions* (i.e. the alternative flows). These fields include pertinent information to the construction of our target RSL models.

Recall that one important step of our approach is the extraction of a semantic model representation of input language-independent use case scenarios. This model was proposed to represent formally raw ideas (i.e., following the way in which we think). This formal representation is independent from the used natural language. Besides, it does not contain operational details (like creating variables, defining their types, methods signatures, etc.). In fact, an idea would be represented always in the same notational way, regardless of the language in which it is originally expressed. For instance, the following sentences "*The course manager is an employee*" and "*The course manager is a kind of employee*" (in English), "*Le gestionnaire de cours est un employé*" (in French), " *Kursmanager ist ein Angestellter*" (in German), "*El administrador del curso es un empleado*" (in Spanish), etc. have the same meaning which is the generalization/specialization between the objects "*course manager*" and "*employee*". In this context, the semantic model analyzes the semantics among these sentences and represents them by one common representation. This model is based on naturalistic features that treat several types of naturalistic entities, whose notations are inspired from [3] [2], namely:

- *Concept*: it is any "*thing*" from the real world, being abstract or concrete.
- *Property*: it describes a concept, e.g. "*short*", "*basic*"...
- *Action*: it represents an activity and is characterized by a number of arguments.
- *Function*: it defines an algorithm whose purpose is the calculation. It is similar to an action but with addition of a return type.
- *Statement*: it is a declarative (part of a) sentence that is either true or false. The semantic model defines five types of statements, namely *possession concept relation, hierarchy concept relation, predicate statement, property statement* and *instance type relation statement*.
- *Sentence*: it is composed of at least two clauses linked by conjunctions (assembling links). A link belongs to a semantic class, such as: (i) *Condition:* uses conditional conjunctions like "*if*", "*in case of*"... (ii) *Contrary:* is the opposite of conditional using "*else*", "*otherwise*"... (iii) *Final:* expresses something happening as a result using conjunctions like "*in order to*", "*to*"... (iv) *Loop:* represents five types of loops, namely: "*for*", "*while*", "*do...while*", "*foreach*" and "*switch*".
- *Referencing*: for example, we say "*Select a course and display its form*"; the word "*its*" is a reference to "*course*". The semantic model accepts several references' types, such as:
  - *Explicit reference:* it allows retrieving a subset of instances from a broader set of instances using the keyword "*the*" in English.
  - *Symbol self reference:* it represents a symbol which refers to a concept, e.g. "*the menu-item 'Attach an object'*"; in this example "*Attach an object*" is a symbol that describes the concept "*menu-item*".
  - *Attribute/relation reference:* it resolves expressions like "*the type of the message*". In fact, this type of expressions contains two other references [2]: the first one placed before the word "*of*" is called a "*filter reference*"; the second one which is placed after the word "*of*" is called a "*collection reference*".
- *Quantity*: natural languages offer an elaborated system of quantification (e.g. "*two students*", "*many information*"...) over instances.
- *Ordinality*: ordinal numbers are used for counting, e.g. "*first*", "*second*"...
- *Compression*: it means grouping different syntactic structures together by some special words like "*and*" (compression type called "*copulative*"), "*or*" (compression type called "*adjunctive*")... It avoids redundancy.
- *Command*: it is a call of an action in the imperative form.

Table 1 illustrates the above entities. Originally, a text is represented in the semantic model in terms of mapping rules to fit the objectives of our previous works [9] [10]. For the purpose of the herein presented approach, we enhanced this model to make it more complete and more exploitable by any approach and tool. Our objective is to structure and implement it in a language specific model in order to have a practical use of its entities. Thus, we decide to represent its entities by *patterns*. Alexander et al. [39] state that "*Each pattern describes a problem which occurs over and over again in our environment, and then describes the core of the solution to that problem, in such a way that you can use this solution a million times over*". This quotation encouraged us to employ patterns in representing the semantic model entities. Originally, the notion of patterns was related to the design patterns (e.g. [40][41]). Then, they were intensively used as a structuring means for other purposes such as security (e.g. [42][43]), business process (e.g. [44][45]), data warehouses (e.g. [46]), etc. In this paper, the semantic model entities are structured in terms of patterns for a practical use in several areas (e.g., automatic code generation, automatic text translation, software product lines, etc.) regardless of the studied domains. In fact, a semantic model pattern includes the following parts:

Table 1
Examples of naturalistic entities.

Entity	Example
Concept	take (a message)$_{concept}$
Property	take a (new)$_{property}$ message
Action	to (send)$_{action_name}$ (something)$_{argument}$ to (someone)$_{argument}$
Function	to (sum)$_{function_name}$ (two integers)$_{arguments}$ is (an integer)$_{return_type}$
Statement	((the mobile phone)$_{argument}$ is (operating)$_{predicate}$)$_{statement}$
Sentence	(the system prompts a message)$_{clause1}$ (in order to)$_{link}$ (inform the user that the message could not be sent)$_{clause2}$
Reference	(it)$_{reference}$ waits for an acknowledgment
Quantity	he sends the e-mail to (many)$_{quantity}$ recipients
Ordinality	the user selects the (first)$_{ordinality}$ contact
Compression	the user fills the (destination, the objects and the body)$_{compression}$ of the e-mail

1. **Pattern name:** gives a label to the pattern.
2. **Intent:** describes the goal of using this pattern.
3. **Structure:** contains the pattern visible structure. This pattern is structured through the *mapping rules* (MRs) representing a language-independent text in a formal and unique way. They are relevant to all natural languages. In other words, a sentence written in different languages will have the same representation, i.e., the same mapping rule. In the presentation of the pattern structure, we will use the EBNF notation [47] as follows:
   - unquoted words denote a non-terminal symbol;
   - quoted words denote a terminal symbol, i.e., a symbol which should be mentioned obligatory;
   - the content of [] is optional;
   - the content of {}– denote symbols repeated one or more times;
   - the character "=" denotes a definition;
   - the semicolon denotes a rule terminator.

   We suppose that CRec-tool stores the mapping rules of a use case scenario (UCS) in th file "*UCS.txt*".
4. **Participants:** describe the entities eventually used within the pattern's structure.
5. **Implementation:** consists in implementing the pattern in XML which is a language specific model that allows expressing clearly the mapping rules. Thanks to its standardization, XML can be easily understood and manipulated by all techniques and tools. Indeed, it has become the de facto language to support data interchange among heterogeneous tools and applications [48]. We suppose that CRec-tool stores the XML implementation of a use case scenario (UCS) within the file "*UCS.xml*".

   In this context, we propose to represent a *concept* by an *object node*, a *property* by an *attribute node*, an *action* by an *operation node* and the effective content of an *input instruction* by an *instruction node*. These XML elements are represented according to the following structure:

   (a) **Declaration of an XML object:** An object has a unique name. It may eventually contain child nodes (attribute(s) and/or operation(s)). It can also be the parent of another object (in case of inheritance). An object node is structured as follows:

```
"<object name="object_name ["parent="parent_object_name]
    ["type="object_type] "id=obj_"id ">"
  [<!-- Eventual attributes list -->]
  [<!-- Eventual operations list -->]
"</object>"
```

   where the strings within double quotes are put as they are, and those which are written between brackets are optionally mentioned. Moreover, the default value of the attribute "*type*" is "*object*"; however, it can hold the values "*system*", "*primary actor*" or "*secondary actor*", as well.

   (b) **Declaration of an XML attribute:** Our approach stores each property in an XML node, called "*attribute*", which contains its name and its type. It conforms to the following structure:

```
"<attribute name="attribute_name "type="attribute_type "id=attr_"id "/>"
```

   (c) **Declaration of an XML operation:** An operation node contains a treatment for the object to which it belongs. It contains two mandatory child nodes referring to the operation's inputs and outputs. When at least one of them is empty, the value "*none*" is assigned to their XML attributes "*name*" and "*type*". Moreover, an operation node may contain eventual instruction nodes that represent its internal treatment. To conclude, an operation is structured as follows:

```
"<operation name="operation_name "id=op_"id ">"
   {"<input name="input_name "type="input_type
      ["role="input_semantic_role] ">" value "</input>"}
   {"<output name="output_name "type="output_type "/>"}
   [<!-- Eventual instructions list -->]
"</operation>"
```

   where the attribute "*role*" refers to the semantic role of the input. By default, this attribute's value holds "*object*".

Table 2
Mapping the semantic model entities to the XML constructs.

XML constructs	SM entities										
	Concept	Property	Action	Function	Command	Statement	Sentence	Referencing	Quantity	Ordinality	Compression
< *object* > < *attribute* >	X	X						X	X	X	X
< *operation* >			X	X							X
< *input* > < *output* >	X	X	X	X				X	X	X	X
< *instruction* > < *result* >					X	X	X				X
< *statement* >						X	X				X

(d) **Declaration of an XML instruction:** The node *instruction* contains an ID, as well as the attribute "*type*" which specifies the type of the instruction. This attribute can hold the following values: (i) "*condition*", i.e. an *If* sentence, (ii) "*contrary*", i.e. an *Else* sentence, (iii) "*exception*", i.e., a call to an alternative flow, (iv) "*reference*", i.e., a reference to the ID of another instruction, (v) "*invocation*", i.e., a call to another use case, (vi) "*final*", i.e., denotes the end of a scenario, (vii) "*loop*", i.e. a kind of loops, (viii) "*action*", i.e. describing an ordinary action. In this latter case, the instruction follows the SVO(O) format as follows:

```
SVO_sentence := [NounPhrase] VerbPhrase
NounPhrase := [Preposition] NaturalisticType
NaturalisticType := [Quantity/Ordinality] [{Adjective}] Noun [Condition]
VerbPhrase := Verb [{NounPhrase}]
```

where the *Naturalistic Type* is defined according to [2]. Otherwise, if this instruction consists of a *condition* or an *exception*, then it contains two child nodes, "*statement*" and "*result*". The former node contains the statement of the condition/exception, whereas the latter node contains a treatment coming as a consequence of the condition/exception. If the instruction type is *contrary*, then it contains the child node(s) "*result*", only. If the instruction type is "*reference*", then its value is the arguments of the reference command. Finally, if the instruction is a *loop*, then it contains an XML node "*nature*", whose value is the type of the loop, and a node "*result*" which contains the treatment intra-loop.

6. **Sample code:** describes some code fragments as an example of the pattern application.

In summary, Table 2 shows how the semantic model entities are mapped to the proposed XML constructs. In this table, the relation between a column element and a row element is "*(may) correspond to*". For example, a concept corresponds to the declaration of the XML node "<object>" and eventually the node "<attribute>" when this concept belongs to another one. Similarly, a property corresponds to the declaration of the XML node "<attribute>" and eventually the node "<object>" when it refers to a noun group. Besides, a concept/property may correspond to the declaration of the XML nodes "<input>" or "<output>" when it consists of an action/function's argument. An action/function corresponds to the XML node "<operation>" and the involved nodes "<input>" and "<output>" that correspond to the action/function arguments. Moreover, the compression may correspond to the creation of *all* the possible XML nodes, depending on the elements involved in this compression. For instance, in case of a compression between concepts, they correspond to the XML nodes "<object>" and eventually "<attribute>".

Due to space limitation, we will present only two patterns, namely the *predicate statement* and the *property assignment* patterns.

Property assignment

1. *Pattern name:* Property assignment
2. *Intent:* Provides the possibility to assign directly a property to a concept using, for example the syntax "*let* type *be* property". A simple property is a property which can be either the case or not, e.g. "*short*", "*basic*", etc. It can be expressed by an adjective which is unchangeable over the time. This kind of properties is defined by assignment.
3. *Structure:* To represent the property assignment formally, we designed the following mapping rule:

```
property assignment= "(property assignment," concept_name "," property_name ")";
```

4. *Participants:*
   - concept_name: designates the concept involved in this pattern.
   - property_name: is the property assigned to the concept.
5. *Implementation:*

```
"<object name="concept_name "id=obj_"id "type="concept_type ">"
 "<attribute name="property_name "type=Boolean id=attr_"id "/>"
"</object>"
"<instruction type=action id=ins_"id ">
     let (" concept_name ") be " property_name "</instruction>"
```

where the *concept_name* (respectively the *property_name*) begins by a capital letter (respectively lowercase letter).

6. *Sample code:* For instance, the following sentence "*Let the message be basic*" conveys to the following mapping rule:

```
(property assignment, (reference, explicit, message), basic)
```

Besides, it is implemented in XML by the following code:

```
<object name="Message" id="obj_1">
 <attribute name="basic" type="Boolean" id="attr_1"/>
</object>
<instruction type="action" id="1">let (message) be basic</instruction>
```

Predicate statement

1. *Pattern name:* Predicate statement
2. *Intent:* This type of statements deals with predicates. A predicate refers to a verb. Besides, it belongs to a class (state or action). Unlike properties, states can change depending on the time, the location, etc.
3. *Structure:* To represent the predicate statement formally, we designed the following mapping rule:

```
Predicate statement= "(statement, (" predicate_class ","
  predicate "),"  ["negation"] { "(" semantic_role "," argument ")" }- ")";
```

where the keyword "*negation*" is put when the statement is in the negative form.
4. *Participants:*
   - predicate_class: is the class of the predicate, being "*action*" or "*state*".
   - predicate: designates the predicate involved in this pattern. It requires a number of arguments, depending on its type (intransitive, transitive or ditransitive [49]).
   - argument: is a predicate's argument; it refers to any constituent that is strictly (semantically) required by the verb. Each argument corresponds to a specific semantic role.
   - semantic_role: is the semantic role of the argument. A *semantic role* is the underlying relation that a constituent has with the main verb in a clause. The following list shows the semantic roles accepted by the semantic model (some of them are present in the literature; some others are proposed by us):
     - *Agent:* the entity that performs the action.
     - *Object:* the entity that undergoes the action.
     - *Comparassant:* a part of a comparison. It designates the compared element.
     - *Comparator:* a part of a comparison. It designates the comparing element.
     - *Possessor:* something that has or contains someone/something.
     - *Possessed:* something that is owned or in the disposal of someone/something.
     - *Sub-concept:* a specialized concept in a hierarchical relationship.
     - *Super-concept:* the generalized concept in a hierarchical relationship.
     - *Origin:* the place or space from where an action is done. It can only occur with action predicates.
     - *Destination:* the place towards which the action is directed.
     - *Location:* the place or space of a predicate expressed by an action or a state.
     - *Time:* indicates the date or the period when an action or a state is done.
     - *Manner:* describes the way of doing something, using the conjunction "*by*" for example.
5. *Implementation:* The predicate statement pattern is implemented by two ways, depending on the class of the predicate: **If** *predicate_class=="state"*, then the state acts as a property for the concept involved in the "object" argument, as follows:

```
"<object name="argument "id=obj_"id ["type="concept_type]">"
 "<attribute name="predicate "type=Boolean id=attr_"id/>
"</object>"
"<instruction type=action id=ins_"id ">
   ("argument") be" predicate"</instruction>"
```

with "*argument*" has the semantic role "*object*".
Else, i.e. *predicate_class=="action"*, then it is interpreted as an operation, as follows:

```
"<operation name="predicate "id=op_"id ">"
 "<input name="argument "type="Argument
    ["role="argument_semantic_role]">" [value] "</input>"
 <!-- here argument's semantic_role="agent" within the mapping rule -->
 [...]
 "<output name=none type=none/>"
```

```
"</operation>"
"<instruction type=action id=ins_"id ">
  ("argument_agent")" predicate ["("other arguments")"]"</instruction>"
```

The semantic model does not recover the exact prepositions which are glued to the concepts in the input scenarios. Thus, we propose the following notation for the arguments within the XML nodes "$<$ *instruction* $>$": The *Origin*, *Location* and *Time* (respectively *Destination* and *Manner*) semantic roles correspond to the particle "*in*" (respectively "*to*" and "*by*").

6. *Sample code:* For instance, the statement "*If the mobile phone is operating, then...*" (in English) or "если мобильный телефон находится в эксплуатации, то..." (in Russian) have equivalent meanings; they are represented by one common mapping rule, as follows:

```
(condition, (statement, (state, be operating),
 (object, (reference, explicit, mobile phone))) [...]
```

Besides, it is implemented in XML by the following code:

```
<object name="MobilePhone" id="obj_"6>
 <attribute name="operating" type="Boolean" id=attr_"5/>
</object>
<instruction type="condition" id="ins_1">
 <statement>(mobile phone) be operating</statement>
[...]
```

We refer the reader to our reference [50] for the exhaustive list of the semantic model patterns. In summary, relying on the developed patterns, the semantic model is able to extract relevant information even from quite complex and ambiguous sentences. For instance, applying the semantic model patterns on the following step of the scenario "*Send message*" from our case study (see Fig. 2): "*The network sends to the system an acknowledgment, which consists of an ASCII character, sent by the recipient of a message (i.e. the destination) back to the user (i.e. the source) in case of the receipt of a recognizable block of data.*" (in English) and "*Le réseau envoie au système un accusé de réception, qui consiste en un caractère ASCII, envoyé par le receveur d'un message (c'est-à-dire la destination) à l'utilisateur (c'est-à-dire la source) en cas de réception d'un bloc de données reconnaissable.*" (in French), we obtain the following common mapping rule:

```
(statement, (action, send), (agent, (reference, explicit, network)),
 (destination, (reference, explicit, system)),
 (object, (acknowledgement, (quantity/ordinality, abstract),
  (property assignment, (reference, relative),
   (ASCII character, (quantity/ordinality, abstract))),
  (condition, (statement, (action, receive),
    (object, (reference, attribute/relation, data, block), recognizable)),
   (statement, (action, send back),
    (agent, (reference, attribute/relation, (message, (quanitity/ordinality, abstract)),
     (reference, explicit, recipient)), (explicative, i.e. the destination)),
    (destination, (reference, explicit, user), (explicative, i.e. the source))))))))
```

Moreover, the following extract of code corresponds to the implementation of the previous mapping rules in XML, based on the semantic model patterns' implementation:

```
<?xml version="1.0" encoding="iso-8859-1"?>
 <object name="Network" type="secondary actor" id="obj_2">
  <operation name="send" id="op_1">
   <input name="system" type="System" role="destination"/>
   <input name="acknowledgement" type="Acknowledgement"/>
   <output name="none" type="none"/>
  </operation>
 </object>
 <object name="Acknowledgement" id="obj_3">
  <attribute name="ASCIICharacter" type="Boolean" id="attr_2"/>
 </object>
 <object name="Recipient" id="obj_4">
  <operation name="sendBack" id="op_2">
   <input name="acknowledgement" type="Acknowledgement"/>
   <input name="user" type="User" role="destination"/>
   <output name="none" type="none"/>
```

```
    <instruction type="condition">
      <statement>(this) receive (recognizable block)</statement>
      ...
    </instruction>
   </operation>
  </object>
  ...
  <instruction type="action" id="5">(network) send (acknowledgement) to (system)</instruction>
  <instruction type="action" id="5.1">let (acknowledgement) be (ASCII character)</instruction>
 ...
```

We refer the reader to section A.1 in the Appendix for more details on the implementation of the mapping rules in CRec-tool. More specifically, our tool creates the mapping rules of each input scenario within its written language. Then, it translates these mappings to the English language by using the Google Translate API (see Section A.1.3 in the Appendix). After getting purely English mapping rules (independently of the input scenarios' language), CRec-tool generates their implementation in XML according to the semantic model patterns (see Section A.2 in the Appendix). In other terms, the generated XML code contain only English words (except for the strings written between quotes in the scenarios and which are considered as *symbols* by the semantic model). Fig. 3 (respectively Fig. 4) shows an extract taken from the generated mapping rules (respectively XML code) for the use case "*Create basic message*" of our case study.

3.2. Redundancy elimination from the obtained XML code

The objective of this step is to eliminate the redundancy among the XML code elements. Because of the natural language aspect of the input scenarios, the latter may use synonymous words to describe the same concept, property, action, etc. For instance, in the context of the GoPhone case study (see Fig. 2), the input requirements contain the semantically-synonymous actions "*select*" and "*choose*" for the concept "*user*". Besides, the corresponding XML nodes "*operation*" have the same inputs and outputs. They are however represented in the generated XML code by two operations, "*select*" and "*choose*", within the same parent object node "*User*". Hence, the XML code (and thus the file "*UCS.xml*" generated by our tool) contains semantic redundancies due to synonyms of XML nodes. To avoid such code redundancy and unify the nodes names, our approach relies on an unsupervised classification, the TF/IDF method [51], for the names within the group of node "*object*" names. This classification is based on their semantics. The grammatical units of a name are extracted according to the standardized notation, i.e. by splitting them according to the capital letters or the character "_" within the names. The TF/IDF method, composed of queries and documents, is based on the calculation of the *cosine similarity measure*. Our approach considers that a query consists of the grammatical units, composing the field "*name*" of an XML node "*object*", and a document is made up of the association of these grammatical units together with their synonyms extracted from *WordNet*[3] [52].

Our approach starts by computing the weight of each grammatical unit, which composes a query q_j and belongs to a document d_i. The weight of each unit is calculated thanks to the following equation [51]:

$$w_{ij} = tf_{i,j} \times idf_{i,j} = tf_{i,j} \times \log(\frac{m}{D(i)}) \tag{1}$$

where: w_{ij} is the weight of the grammatical unit i in the document j; $tf_{i,j}$ is the frequency of the unit i in the document j; m is the total number of documents in the collection; and $D(i)$ is the number of documents where the unit i occurs. After that, our approach computes the cosine similarity $Sim(d_i, q)$ between a document d_i and a query q, using the following equation [51]:

$$Sim(d_i, q) \approx \cos(\vec{d_i}, \vec{q}) = \frac{\sum_{t_j \in U} w_{ij} \times w_{qj}}{\sqrt{\sum_{t_j \in U} w_{qj}^2 \times \sum_{t_j \in U} w_{ij}^2}} \in [0, 1] \tag{2}$$

where: w_{ij} is the weight of the grammatical unit i in the document j; w_{qj} is the weight of the unit j in q; and U is the set of grammatical units composing all the documents. After performing this calculation, our approach rearranges the couple of names having the highest values of this measure into one semantic class. Then, it selects a name for each class and it replaces the fields "*name*" of the XML nodes "*object*" by the selected semantic class names. Our approach applies the same process on each group of "*operation*" nodes and "*attribute*" nodes that belong to the same "*object*" node. Finally, it generates a refined standardized XML code, ready to be transformed to the inputs of an existing code generator. This process was implemented within CRec-tool which eliminates the redundancy from the content of the file "*UCS.xml*" (see Section A.3 in Appendix A).

[3] WordNet is used only for the classification task, independently of the mapping rules synthesis process. Indeed, CRec-tool generates the mapping rules by using predefined NOOJ dictionaries, NOOJ syntactic/semantic grammars and the Google translate API (see section A.1.3 in the Appendix).

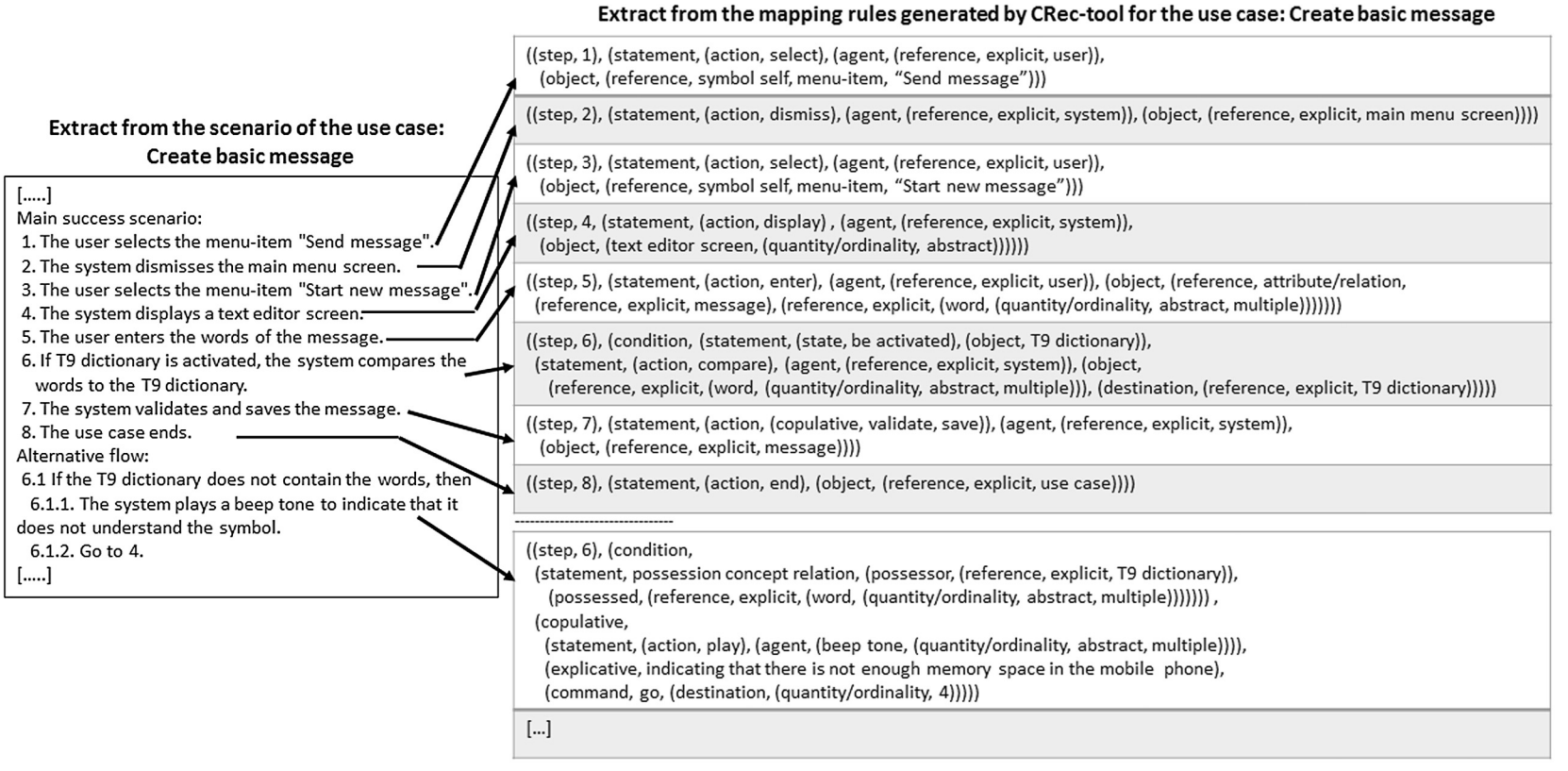


Fig. 3. An extract from the generated mapping rules for the use case "Create basic message" of our case study.

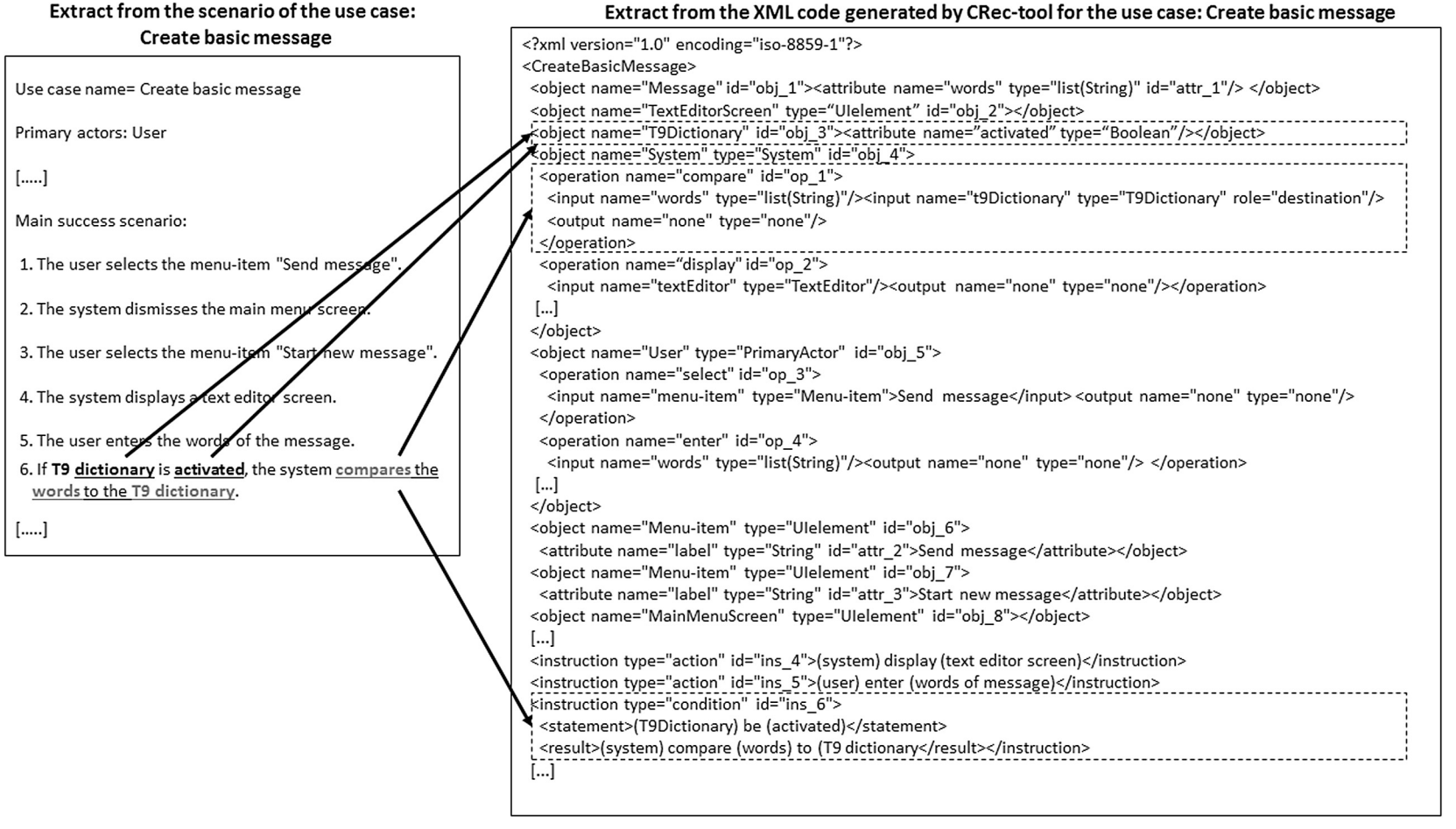


Fig. 4. An extract from the generated XML code for use case "Create basic message" of our case study.

3.3. Converting the XML code to the entries required by the ReDSeeDS generator

At this stage, our approach is capable of transforming the resulting improved XML code into the syntax of existing code generators, such as ReDSeeDS [8]. The latter allows generating a source code in Java from requirements by following a MVP architectural pattern. Indeed, ReDSeeDS reduces the need for direct programming in Java, as well as the role of programmers to update the code generated by this code generator [53].

ReDSeeDS is based on the *requirements specification language* (RSL) which determines how to denote various phrases in order to generate good quality architectures of the target applications [53]. The RSL language enables the specification of the *requirements model* and the *domain model* [18]. We refer the reader to [18] for a complete description of these models. Thus, our approach converts the XML code to RSL and produces the requirements and domain models. In the remainder of this section, we will explain the process followed by our approach to build these RSL models. More specifically, we will propose two algorithms whose application on the XML code built by our approach, generates the corresponding ReDSeeDS inputs (i.e. the RSL models).

3.3.1. RSL requirements model

ReDSeeDS considers the RSL use cases as the most special type of requirements. Indeed, they are pieces of observable functionality that lead to goals of some value to outside actors [18]. In this context, we identify two RSL constructs: (i) the RSL use cases and their relationships, as well as (ii) the RSL use case contents, i.e. the scenarios sentences.

Concerning the first construct, the notation of the RSL use cases and their relationships are inspired from UML with some specificities. Indeed, the RSL use cases can be related to each other or to the actors by the following relationships:

- ≪*use*≫: denotes the initiation of an actor (considered as a primary actor) to the execution of a use case.
- ≪*participate*≫: when a particular actor (considered as a secondary actor) is called and contributes in the realization of the actual RSL use case.
- ≪*invoke*≫: substitutes the UML "*include*" and "*extend*" relationships between UML use cases.

Each RSL use case should have at least one representation as a "*Constrained Language Scenario*" (CLS) [53]. The latter is composed of the steps of the UML use case scenario. In this context, RSL uses a controlled natural language, in English, to describe these steps. According to the CLS, a scenario sentence can be a precondition, an SVO, a condition, an invocation, a final, a postcondition or a rejoin sentence [53][18]:

- a *Precondition*: starts by the keyword "*Pre:*" followed by a free text that describes the actual condition. This sentence provides the execution condition of the RSL use case. In the current version of ReDSeeDS, the involved code is not generated; in fact, it has to be updated manually.
- an *SVO Sentence*: defines individual actions. It is a sentence composed of a *Subject* and a *Predicate*. This predicate is made of a verb, a direct object and possibly an indirect object. In other terms, verbs are assigned to sentence objects. In addition, the subject of this sentence could be only a noun phrase. In other terms, there is no use of *naturalistic references*. *Quantities* and *ordinalities* are not supported by RSL. The subject and the predicate are hyperlinks (i.e., references) to definitions of the involved phrases within the domain model (see Section 3.3.1). An hyperlink consists of linked term names preceded by a letter with a colon (":") indicating the term type ("n:" for a *NounLink*, "m:" for a *ModifierLink*, "v:" for a *VerbLink*, "p:" for a *PrepositionLink*). Besides, it is surrounded by a pair of square brackets [8].
- a *Condition sentence*: starts with the keyword "->*cond:*" followed by the actual condition. When this condition is met, the flow control goes to the sentence that follows the condition. Otherwise, the flow control moves to another scenario. More specifically, the presence of a condition group, namely an alternative flow, leads to having one scenario per condition.
- an *Invocation sentence*: starts with the keyword "->*invoke:*" followed by the name of the specific invoked use case. This sentence triggers and inserts the content of the invoked RSL scenario.
- a *Final sentence*: starts with the keyword "->*final:*" followed either by the "*success*" or the "*failure*" keywords. Whenever such a sentence is reached, the use case terminates its execution and passes the control to where it was called from.
- a *Postcondition*: starts by the keyword "*Post:*" followed by a free text describing the state in which the system should end.
- a *Rejoin sentence*: starts with the keyword "->*rejoin:*" followed by the rejoin point, i.e., the SVO sentence at which the scenario has to be rejoined.

Note that whenever an SVO sentence is created, its parts should be hyperlinked to appropriate phrases in the RSL domain model. The following section deals with the description of this model.

3.3.2. RSL domain model

The generation of Java code requires the specification of the RSL requirements model and the RSL domain model. The specification of the latter involves the definition of the *actors*, the *domain elements* and the *application domain*.

Similar to the UML notation, an RSL actor is that who wants to reach the final goal associated with the use case [18].

The domain elements consist in notions. A notion is a (set of) phrase composed of nouns which describe a domain entity. A phrase is composed as follows:

```
Phrase := NounPhrase | VerbPhrase
NounPhrase := [Modifier] + Noun
VerbPhrase := SimpleVerbPhrase | ComplexVerbPhrase
SimpleVerbPhrase := NounPhrase + Verb
ComplexVerbPhrase := NounPhrase + SimpleVerbPhrase
```

A notion can be a *concept* (resembling to an UML class), an *attribute* (holding elementary data with its type), or a *data view* (presenting an instance) [18]. The domain rules (i.e. the data processing) are organized through actions included in the notions. An action consists, at least, of a verb and a noun. In other terms, verbs are namely attached to the sentence objects. In fact, the object which is directly related to the verb reflects the actual notion containing a particular action. For instance, all the actions with the direct object "message" are contained in the notion "Message". There exist some actions predefined in RSL, such as SHOW (used to show the data view attributes values, e.g., "show", "present", etc.), CLOSE (removes the UI element from the user interface, e.g., "close", "shut", etc.), CREATE (adds new data items to the system's storage, e.g., "create", "save", etc.)...

Concerning the application domain, it groups the UI elements. Each UI element belongs to one of the following types [18]:

- Screen: it is an element presented to the users who can interact with it. It contains various data elements.
- Trigger: it is associated with interactions of the user with the system, like buttons, menu items, etc.
- Messages and Confirmations: they offer simple functionality for presenting information and accepting decisions from the user.

3.3.3. Extracting RSL models from XML code

In this section, we will propose two algorithms applied to extract ReDSeeDS entries from the improved XML code generated by our approach (see Section 3.2). The extraction of Java code using this generator necessitates the specification of the RSL requirements model and the RSL domain model.

It is worth mentioning that there is neither an API nor a web service enabling the communication of external applications with ReDSeeDS. In other terms, our tool cannot establish a connection to this generator. Indeed, we have to put and to execute manually the RSL models generated by our approach within ReDSeeDS in order to get the target Java code. As a perspective (see the conclusion section), we will try to build an API enabling the insertion of the RSL models generated by our approach within the ReDSeeDS repository. In this context, our approach generates the XML file "*RequirementsModel.xml*", containing the specification of the RSL requirements model, according to the following structure:

```
<?xml version="1.0" encoding="iso-8859-1"?>
 <RequirementsModel>
   <UseCase id="UseCase_id" name="UseCase_name"
     parent="UseCase_parent_id">
      <UseRelationship actor="using_actor_name"/>
      <ParticipateRelationship actor="participating_actor_name"/>
      <InvokeRelationship usecase="invoked_UseCase_name"/>
      <Scenario url="rsl_file.rsl"/>
   </UseCase>
  ......
  </RequirementsModel>
```

In this context, our approach relies on Algorithm 1 to map the XML code generated by our approach to the constructs of the RSL scenarios, according to the CLS language (see Section 3.3.1). CRec-tool stores the RSL code in RSL files.

Our approach is also based on the use case scenarios and the resulting RSL files in order to build the XML hierarchy of the file "*RequirememntsModel.xml*"; this file is used as an input to the ReDSeeDS generator to extract the target Java code. The following extract of XML code is generated by CRec-tool for our case study based on the guidelines in Algorithm 1:

```
<?xml version="1.0" encoding="iso-8859-1"?>
<RequirementsModel>
  <UseCase id="1" name="Create basic message" parent="-1">
     <UseRelationship actor="user"/>
     <InvokeRelationship usecase="Attach object"/>
     <InvokeRelationship usecase="Send message"/>
     <Scenario url="create_basic_message.rsl"/>
     <Scenario url="create_basic_message1.rsl"/>
```

Algorithm 1: Extraction of the RSL requirements model from the generated XML code.

Data: "UCD.xml"
Result: RSL files
Create the file "*UCD.rsl*" /* It contains the main success scenario steps in RSL */
Put a Precondition sentence in the file "*UCD.rsl*" according to the following syntax: $Pre : +Precondition$ /* i.e., the content of the field "Precondition" in the use case scenario */
foreach *independent XML node instruction,* ins_i*, in the file "UCS.xml"* **do**
  /* The scenario sentences are put in the file $UCD.rsl$ (respectively $UCS_k.rsl$) in case of the main success scenario (respectively alternative flow) */
  switch $ins_{i,type}$ **do**
    case *action* **do**
      /* ----- Bloc 1: */
      Replace the strings delimited by parentheses between "[n:" and "]" in ins_i
      Put the verbs between "[v:" and "]" in ins_i
      Put the prepositions between "[p:" and "]" ins_i
      Remove the parentheses from ins_i
      /* ----- End of Bloc 1 */
      Put $ins_{i,value}$ in the file "*UCS.rsl*"
    case *condition* **do**
      foreach *XML node statement,* st_j*, with* $st_j \in ins_i$ **do**
        Perform Bloc 1 by changing the variable ins_i with st_j
        Put a condition sentence in the file "*UCS.rsl*", by following this syntax: $-> cond : +st_j$
      end
      foreach *XML node result,* res_j*, with* $res_j \in ins_i$ **do**
        Perform Bloc 1 by changing the variable ins_i with res_j
        Put res_j in the file "*UCS.rsl*"
      end
    case *contrary* **do**
      Put the latest mentioned condition, *lastcond*, in the file "*UCS.rsl*", by following this syntax: $-> cond : +NOT + lastcond_{statement}$
      foreach *XML node result,* res_j*, with* $res_j \in ins_i$ **do**
        Perform Bloc 1 by changing the variable ins_i with res_j
        Put res_j in the file "*UCS.rsl*"
      end
    case *exception* **do**
      Create the file "UCS_k.rsl"
      Select the content of the file "UCS.rsl" from its beginning to the step that precedes the step that raised the exception
      Copy the selected sentences in "UCS_j.rsl"
      foreach *XML node statement,* st_j*, with* $st_j \in ins_i$ **do**
        Perform Bloc 1 by changing the variable ins_i with st_j
        Put a condition sentence in the file "*UCS.rsl*", by following this syntax: $-> cond : +st_j$
      end
      foreach *XML node result,* res_j*, with* $res_j \in ins_i$ **do**
        Perform Bloc 1 by changing the variable ins_i with res_j
        Put res_j in the file "$UCS_k.rsl$"
      end
    case *reference* **do**
      Put a rejoin sentence in the file "*UCS.rsl*", by following this syntax: $-> rejoin : +ins_{i,value}$
  end
end
case *invocation* **do**
  Put an invocation sentence in the file "*UCS.rsl*", by following this syntax: $-> invoke : +ins_{i,value}$
  /* or the file "$UCS_k.rsl$" in case of an alternative scenario */
case *final* **do**
  if $ins_i \in$ field "*Description*" **then**
    /* i.e. it belongs to the main success scenario */
    Put the following sentence in the file "*UCD.rsl*": $-> final : success$
  else
    Put the following sentence in the file "*UCD_j.rsl*": $-> final : failure$
  end
Put a Postcondition sentence in the file "*UCD.rsl*" according to the following syntax: $Post : +Postcondition$; /* i.e., the content of the field "Postcondition" in the use case scenario */

```
    <Scenario url="create_basic_message2.rsl"/>
    <Scenario url="create_basic_message3.rsl"/>
    <Scenario url="create_basic_message4.rsl"/>
    [...]
  </UseCase>
  <UseCase id="1" name="Send message" parent="-1">
    <UseRelationship actor="user"/>
    <ParticipateRelationship actor="network"/>
    <Scenario url="send_message.rsl"/>
```

```
    [...]
  </UseCase>
 [...]
 </RequirementsModel>
```

Fig. 5 shows the content of the file "*create_basic_message.rsl*" and the RSL files of the alternative flows for this scenario. More specifically, this figure demonstrates how the alternative flows of the scenario "*Create basic message*" (see steps 6.1, 7.1, 7.2 and 8.1 in Fig. 5, on the left side) are transformed into the corresponding RSL scenario files: "*create_basic_message1.rsl*", "*create_basic_message2.rsl*", "*create_basic_message3.rsl*" and "*create_basic_message4.rsl*" (see Fig. 5, on the right side).

On the other hand, our approach generates the XML file "*DomainModel.xml*", containing the specification of the RSL domain model, according to the following structure:

```
<?xml version="1.0" encoding="iso-8859-1"?>
<DomainModel>
  <Actor id="actor_id" name="actor_name" type="actor_type"/>
  <DomainElement id="notion_id" name="notion_name" type="notion_type">
    <description>[...]</description>
    <attribute name="attribute_name" type="attribute_type"/>
    <action name="action_name">
      <parameter name="parameter_name"/>
    </action>
  </DomainElement>
 <ApplicationDomain id="UI_id" name="UI_name" type="UI_type">
    <action name="action_name">
      <parameter name="parameter_name"/>
    </action>
  </ApplicationDomain>
  ......
</DomainModel>
```

To this end, our approach relies on Algorithm 2 for extracting the target RSL domain model.

For instance, the following extract of XML code is generated by our approach; it is deduced from the scenario of the use case "Create basic message" (see Fig. 5, on the left side) and the corresponding RSL code (see Fig. 5, on the right side) by following the guidelines of Algorithm 2:

```
<?xml version="1.0" encoding="iso-8859-1"?>
<DomainModel>
  <Actor id="1" name="user" type="primary actor"/>
  <Actor id="2" name="system" type="system"/>
  [...]
  <DomainElement id="2" name="message" type="concept">
    <attribute name="word" type="String"/></DomainElement>
  <DomainElement id="3" name="words" type="data_view">
    <action name="enter"><parameter name="words"/></action>
    <action name="compare">
      <parameter name="words"/>
      <parameter name="T9 dictionary"/></action>
  </DomainElement>
  <DomainElement id="4" name="T9 dictionary" type="concept">
    <attribute name="activated" type="data_view"/></DomainElement>
  [...]
  <ApplicationDomain id="UI1" name="Send message menu-item" type="trigger">
    <action name="select"><parameter name="Send message menu-item"/></action>
  </ApplicationDomain>
  <ApplicationDomain id="UI2" name="main menu screen" type="screen">
    <action name="close"><parameter name="main menu screen"/></action>
  </ApplicationDomain>
  <ApplicationDomain id="UI3" name="text editor screen" type="screen">
    <action name="show"><parameter name="text editor"/></action>
  </ApplicationDomain>
  <ApplicationDomain id="UI4" name="Start new message menu-item" type="trigger">
    <action name="select"><parameter name="Start new message menu-item"/></action>
  </ApplicationDomain>
  [...]
</DomainModel>
```

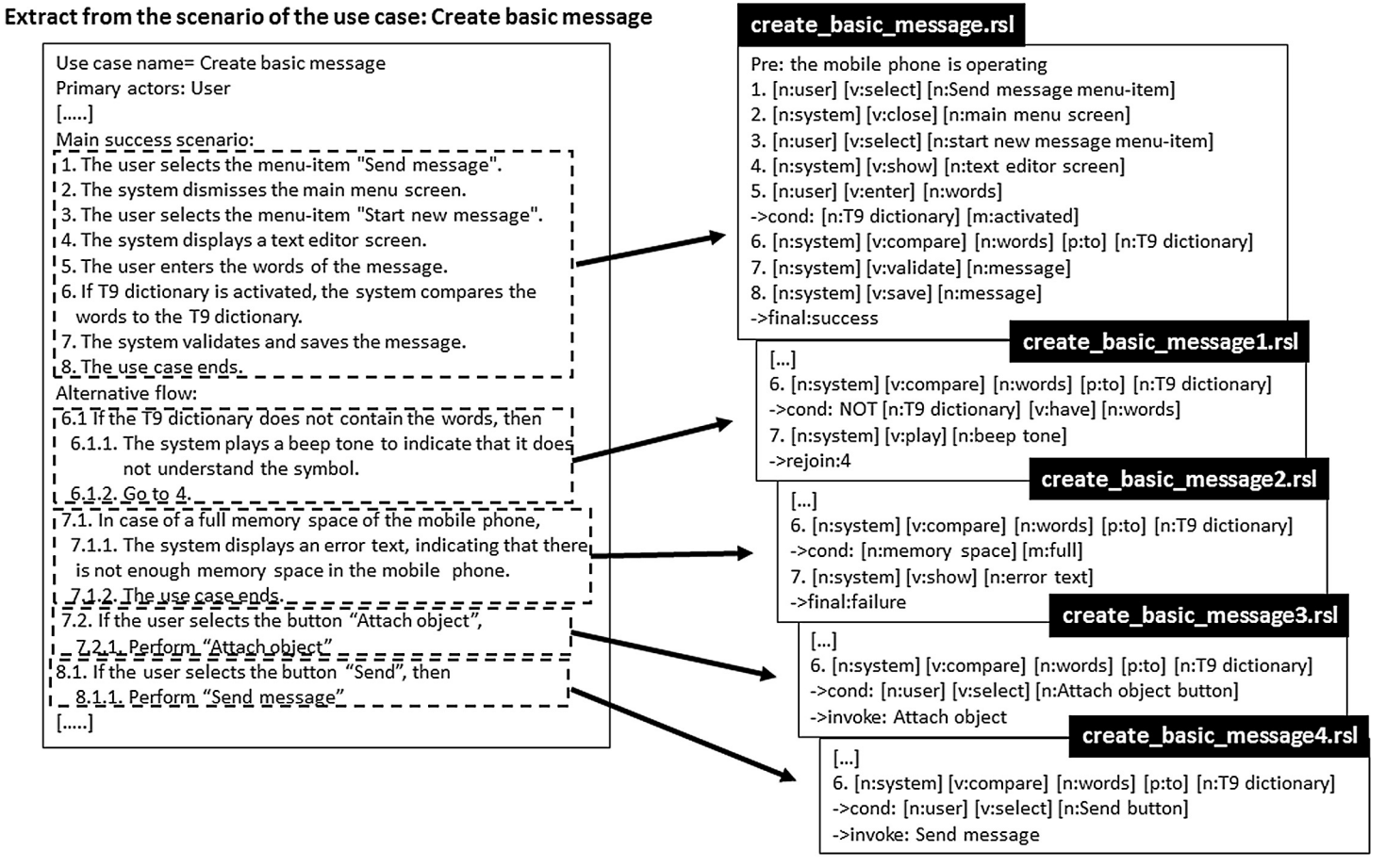


Fig. 5. An extract from the generated RSL files.

Algorithm 2: Extraction of the RSL domain model from the generated XML code.

Data: "*UCD.xml*"
Result: "*RSLModel.xml*"
foreach *XML node object, obj_i, in the file* "UCS.xml" **do**
  switch $obj_{i,type}$ **do**
    case *"actor"* **do**
      Create the actor, $actor_i$, in the file "*DomainModel.xml*", with: $actor_{i,name} \leftarrow obj_{i,name}$ and $actor_{i,type} \leftarrow obj_{i,type}$
    case *"object"* **do**
      Create the domain element, $notion_i$, in the file "*DomainModel.xml*", with: $notion_{i,name} \leftarrow obj_{i,name}$ and $notion_{i,type} \leftarrow concept$
      foreach *XML node operation, op_j, with $op_j \in obj_i$* **do**
        If ∄, create the XML node *action*, *act*, in the domain element obj_i within the file "*DomainModel.xml*", with: $act_{name} \leftarrow op_{j,name}$
        foreach *XML node input, in_k, with $in_k \in act$* **do**
          Create the XML node *parameter*, *param*, in the action *act* within the file "*DomainModel.xml*", with: $param_{name} \leftarrow in_{k,name}$
          if $in_{k,role}$ == *"object"* **then**
            If ∄, create the XML node *action*, *act*, in the domain element in_k within the file "*DomainModel.xml*", with: $act_{name} \leftarrow op_{j,name}$
            foreach *XML node input, $in_l \in op_j$* **do**
              Create the XML node *parameter*, *param*, in the action *act* within the file "*DomainModel.xml*", with: $param_{name} \leftarrow in_{l,name}$
            end
          end
        end
      end
    case *"system"* **do**
      Create the actor, $actor_i$, in the file "*DomainModel.xml*", with: $actor_{i,name} \leftarrow obj_{i,name}$ and $actor_{i,type} \leftarrow obj_{i,type}$
      foreach *XML node operation, $op_j \in obj_i$* **do**
        foreach *XML node input, $in_k \in op_j$* **do**
          switch $in_{k,type}$ **do**
            case *list* **do**
              If ∄, create the domain element, $notion_i$, in the file "*DomainModel.xml*", with: $notion_{i,name} \leftarrow in_{k,name}$ and $notion_{i,type} \leftarrow data_view$
            case *UIElement* **do**
              If ∄, create the application domain, UI_i, in the file "*DomainModel.xml*", with: $UI_{i,name} \leftarrow in_{k,name}$ and $UI_{i,type} \leftarrow in_{k,value} + in_k$
          end
        end
      end
    case *"UIelement"* **do**
      If ∄, create the application domain, UI_i, in the file "DomainModel.xml", with: $UI_{i,name} \leftarrow value\ of\ UI_{i,attribute} + obj_{j,name}$
  end
end
foreach *XML node attribute, $attr_j$, with $attr_j \in obj_i$* **do**
  if $attr_{j,type} == Boolean$ **then**
    If ∄, create the XML node *attribute*, *attr*, in the domain element obj_i within the file "DomainModel.xml", with: $attr_{name} \leftarrow attr_{j,name}$ and $attr_{type} \leftarrow data_view$
  else
    If ∄, create the XML node *attribute*, *attr*, in the domain element obj_i within the file "DomainModel.xml", with: $attr_{name} \leftarrow attr_{j,name}$ and $attr_{type} \leftarrow attr_{j,type}$
  end
end

This process is implemented within the CRec-tool (see Section A.4 in the Appendix). In conclusion, the generated XML files, "*DomainModel.xml*" and "*RequirementsModel.xml*", serve as the entries to the ReDSeeDS tool.

3.4. Java code generation with the ReDSeeDS generator

To generate the Java code corresponding to the resulting RSL models, ReDSeeDS focuses basically on the SVO sentences within the requirements model. These sentences define the individual interactions between the actor and the system, and system responses; they can be additionally divided into three types [29,18], as follows:

- *Actor-to-system*: the subject points to an actor, and the direct object points to a user interface element or to a data view (e.g., see sentences 1, 3 and 5 in the file "*create_basic_message.rsl*" within Fig. 5). A scenario should start by such a sentence. Besides, this latter should follow any system-to-actor sentence.
- *System-to-actor*: the subject points to the system, and the direct object points to a user interface element (see sentences 2 and 4 in Fig. 5). Such sentences can follow any other sentence in a scenario.
- *System-to-system*: the subject points to the system, and the direct object points to a domain element (e.g., see sentences 6, 7 and 8 in the file "create_basic_message.rsl" within Fig. 5). Such sentences can follow only actor-to-system and other system-to-system sentences.

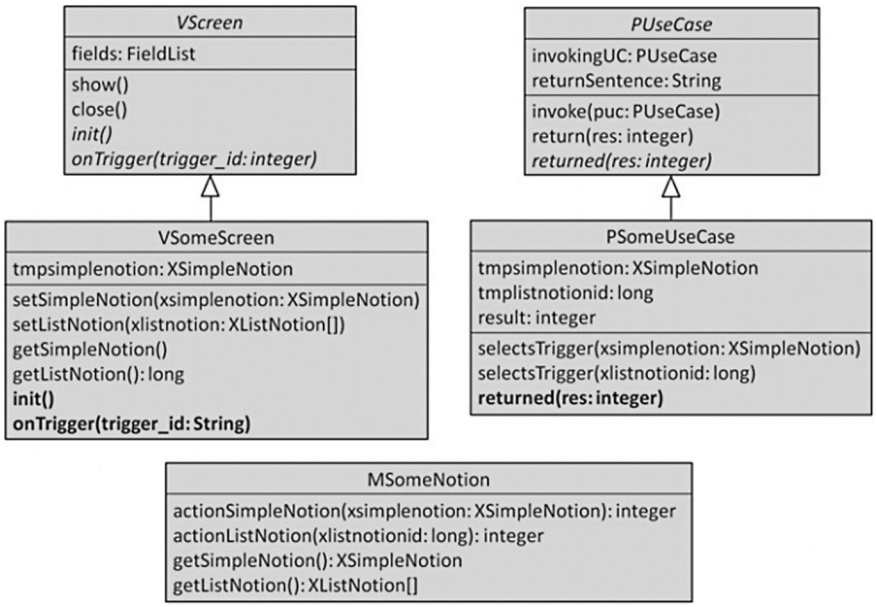


Fig. 6. Structure of the target MVP classes [18].

The target MVP classes, generated by ReDSeeDS, have a specific structure as illustrated in Fig. 6 [18]. This figure shows namely the class "*XSimpleNotion*" (its name starts by the prefix "X"). This special class is called a Data Transfer Object (DTO). The instances of such classes are used to pass data between layers (model, view and presenter layers). Moreover, we note that each view class ("*VSomeScreen*") specializes from the class "*VScreen*". This latter class contains the following elements:

- the attribute "*fields*": an ordered list of data fields.
- the concrete method "*show()*": opens a new window, renders all the fields and presents them to the user.
- the concrete method "*close()*": removes the window associated with the current View elements.
- the abstract method "*init()*": contains the code that defines all the fields in a given window.
- the abstract method "*onTrigger()*": called when an event occurs within the user interface. It contains one parameter, "*trigger_id*", which determines the actual trigger that was selected.

Similarly, each presenter class ("*PSomeUseCase*") specializes from the class "*PUseCase*". This latter class defines the following elements:

- the attribute "*returnSentence*": remembers the context in which the invocation was performed.
- the attribute "*invokingUC*": used by the invoked Presenter class instance.
- the concrete method "*return()*": returns the control to this remembered invoking instance.
- the abstract method "*returned()*": returns the control of the invoking instance. It contains the parameter "*res*", which passes the final state of the invoked use case.
- Implementing the event handlers; these latter respond to the triggers captured by the View classes.

ReDSeeDS follows some transformation rules on the RSL models [18,29], among which we cite the following ones:

- For each use case, a *presenter* layer class is created with the prefix "P". This class specializes from the class "*PUseCase*". Moreover, it contains the operations involved within: (i) the *actor-to-system* sentences of the scenario, (ii) the *invocation sentences*, and (iii) the appropriate associations (i.e., the "*returned*" operation).
- For each UI element, a *View* layer class is created with the prefix "V". This class contains the operations involved within the *system-to-actor* sentences.
- For each notion, a *Model* layer class is created with the prefix "M". The eventual involved attributes and actions are added to this class. Besides, this latter contains the operations involved within the *system-to-system* sentences.
- Every *actor-to-system* sentence directly following a *system-to-actor* sentence is transformed into a button declaration within the "V" class containing the method where a call to this method was generated, as well as an associated event handler function.
- Each *InvocationSentence* corresponds to an operation, whose name is the concatenation of the phrase "*invoke*" and the invoked use case name.

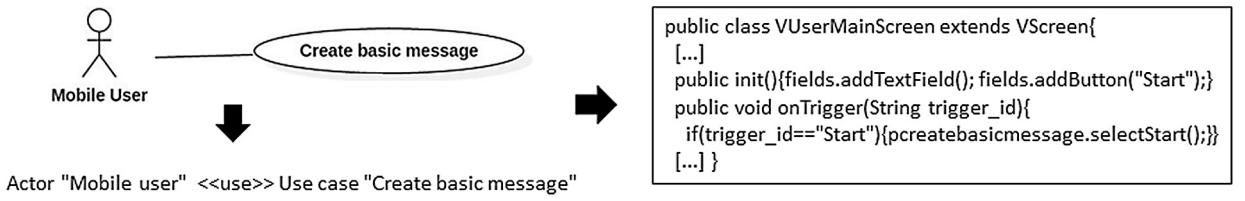


Fig. 7. An extract from the generated Java code for our case study -1-.

- Every ≪*use*≫ relationship between an actor and a use case is semantically equivalent to an invoke relationship with an additional invoking Use Case. The actor is substituted by an additional Use Case which becomes the "main" use case which is invoked at the start of the application.
- For each *SVOSentence* that precedes an InvocationSentence, if it consists of a *system-to-actor* or a an *actor-to-system* sentence, then (i) the invocation related code is generated in the current "V" class, and (ii) this code contains a button generation, an event handling and a call to the appropriate "*invoke*"+$UseCase_{name}$ operation.

Applying these transforation rules on the RSL models generated by our approach (and thus our tool) leads ReDSeeDS to generate the corresponding Java code, which follows the MVP architecture. Figs. 7 and 8 show two extracts from some of the resulting Java code for our case study.

4. Evaluation

To evaluate our approach and the CRec-tool, we applied the structure proposed by Wohlin et al. [54], who decomposed the evaluation step into different parts, such as *goal, task, subjects, preparation, conduction* and *evaluation*. In the remainder of this section, we will evaluate the ability of our approach, and thus our tool, to build RSL domain models and requirements models, conform to the experts ones. Secondly, we will assess the efficiency of the Java code generated by our approach by relying on programmers' feedback. Thirdly, we will evaluate the ability of our method to extract the XML code elements conform to the input requirements based on students' feedback. Finally, we will discuss some threats to validity of our approach.

4.1. Evaluating the ability of our approach to extract the RSL models conforming to the experts' ones

Goal: Śmialek et al. [29] evaluated the ReDSeeDS tool. The results showed clearly that ReDSeeDS enhances significantly the productivity of programming. However, it is useful to evaluate the conformity extent between the domain models and requirements models, generated by our tool, and those built by experts starting from the same inputs.

Subject and Preparation: To evaluate the quality of the ReDSeeDS inputs generated by our tool, we gave two RSL experts (natural language processing PhD students who are used to manipulate the ReDSeeDS generator) the following scenarios list belonging to five different systems:

- 16 use case scenarios belonging to a *health complaint application* [55]; this latter allows citizens to report complaints (food, animal and special complaints) via internet.
- A well developed scenario of the use case "*Withdraw cash*" belonging to a *banking system* [56].
- 10 use case scenarios belonging to a *crisis management system* [57]; it treats crisis, which can range from major to catastrophic affecting many segments of society.
- 12 use case scenarios belonging to a *campus management system* [58] [59]; it is an application for the management of students data.
- 5 use case scenarios belonging to *Emptio* [60], which is a mobile phone application for self-service shopping.

The scenarios used in our evaluation were taken from the above-cited references of the afore-mentioned systems. In other terms, they are supposed to be built by experts. Besides, our experiments are independent of the form of the used scenarios. The most important thing is that the main success scenarios and the alternative flows are written in natural language.

Some of the scenarios were written in English, while others were in French. Evidently, the two RSL experts had good knowledge of these two languages. Besides, these requirements had quite complex and ambiguous sentences' structures.

Task: We asked the RSL experts to give us the correct RSL domain models and RSL requirements models corresponding to the adopted subjects.

Conduction: We compared the experts' RSL models to the corresponding ones generated by our tool by using the *precision* (see equations (3)), the *recall* (see equations (4)), and the *F-measure* (see equations (5)) metrics in terms of RSL domain models (respectively pDM, rDM and FDM) and RSL requirements models (respectively pRM, rRM and FRM).

$$Precision = \frac{TP}{TP + FP} \times 100 \tag{3}$$

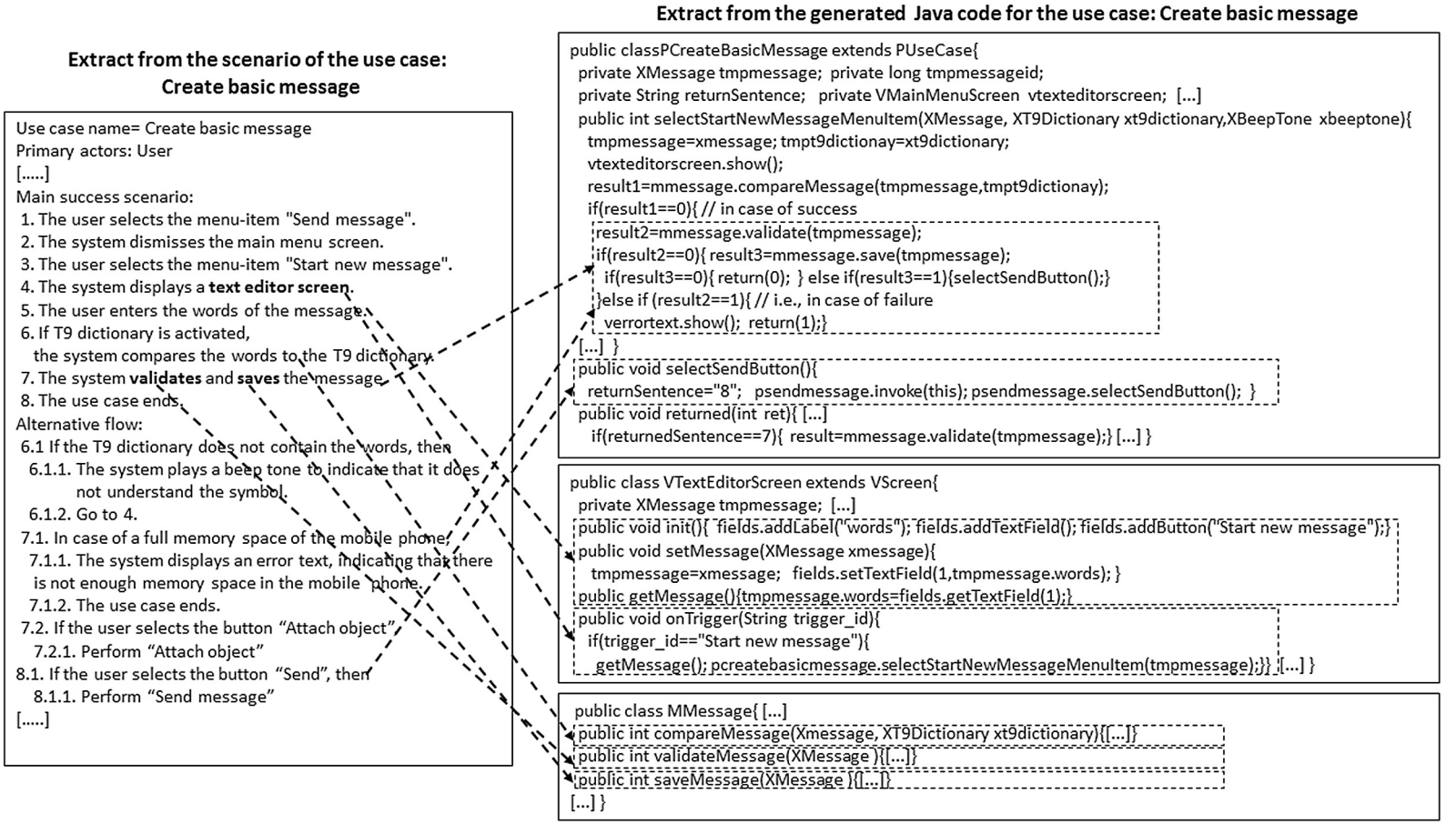


Fig. 8. An extract from the generated Java code for our case study -2-.

Table 3
Evaluation of the ReDSeeDS inputs generated by our tool.

System	pDM	rDM	FDM	pRM	rRM	FRM	ComDM	CorDM	ComRM	CorRM
Campus management	83.78%	93.94%	88.57%	85%	91%	87.9%	93.55%	96.77%	94.11%	93.32%
Banking	80.95%	94.44%	87.18%	93%	93%	93%	94.12%	90.59%	95.45%	94.13%
Health complaint	78.57%	91.67%	84.62%	90%	93%	91.48%	90.91%	93.94%	91.04%	89.86%
Crisis management	78.38%	90.63%	84.06%	87%	90%	88.47%	89.66%	89.66%	97.14%	92.78%
Emptio	80.77%	91.3%	85.71%	89.66%	89.66%	86.21%	90.48%	95.24%	94.76%	89.91%
Average	**80.49%**	**92.4%**	**86.03%**	**89%**	**91%**	**90.1%**	**91.74%**	**92.55%**	**94.5%**	**92%**

$$Recall = \frac{TP}{TP + FN} \times 100 \tag{4}$$

$$F - measure = \frac{2 \times Precision \times Recall}{Precision + Recall} \tag{5}$$

where TP (*true positives*) is the number of true RSL models' elements retrieved by our tool; FP (*false positives*) is the number of RSL models' elements generated by our tool but absent in the experts' RSL models; FN (*false negatives*) is the number of pertinent RSL models' elements, which are absent in the RSL models built by our tool. Moreover, we decided to use two other metrics taken from the standard ISO 25020. in order to measure:

- The *completeness* (Com) degree of our results in comparison to the RSL experts' ones; for example, it is measured, in terms of domain models, as follows:

$$ComDM = 1 - \frac{number\ of\ unfound\ pertinent\ domain\ model's\ elements}{number\ of\ pertinent\ domain\ model's\ elements} \times 100 \tag{6}$$

- The *correctness* (Cor) of our results in comparison to the RSL experts' ones; for instance, it is computed, in terms of domain models, as follows:

$$CorDM = \frac{number\ of\ domain\ model's\ elements\ adequately\ implemented}{number\ of\ pertinent\ domain\ model's\ elements} \times 100 \tag{7}$$

We note that the number of domain/requirements model's elements adequately implemented means that the majority of their implementations are pertinent according to the experts' implementations.

Table 3 shows the resulting values of the adopted measurements.

Evaluation: Table 3 shows the high average values of the adopted metrics, which exceed 80% in all cases. More specifically, the precision (respectively recall) values in terms of domain models range from 78.38% to 83.78% (respectively 90.63% to 94.44%). Similarly, we note high values of completeness and correctness, reaching respectively 94.12% and 96.77%. For example, in the case of the campus management system, we obtained the highest values of precision in terms of RSL domain models (83.78%). This fact means that our tool generates a good number of *true positives*, which equals 31. vs. a low number of *false positives*, which equals 6. Moreover, the recall values in terms of RSL domain models for the campus management system equals 93.94%, reflecting that our tool generates the majority of pertinent domain models' elements. The high values of completeness (92.55%) and correctness (96.77%) confirm this quotation.

We note that our tool generates some false positives which decreased the domain models' precision values in some cases. For example, in the case of the health complaint system, our tool generates 33 true positives, vs. 9 false positives; in fact, the use case scenarios of this system treat each *type* of a *complaint query* on its own. Thus, our approach generates four notions corresponding to four queries' types: "*onSpecialities*", "*onHealthUnits*", "*onDiseases*" and "*onComplaint*". In fact, the input scenarios do not contain any information allowing our tool to recognize, automatically, the notion "*type*", which corresponds to the above four values. However, the expert realizes that these notions correspond to only one RSL attribute "*type*", which corresponds to an *anonymous concept* (i.e. a concept which should contain no other concepts, nor actions, nor functions) with four possible values: "*onSpecialities*", "*onHealthUnits*", "*onDiseases*" and "*onComplaint*" within the RSL notion "*Query*". In contrary, our approach generates one attribute instead of a notion, entitled "*complaintType*" within the RSL notion "*Complaint*" and admits it as an anonymous concept with four possible values: "*food*", "*special*", "*animal*" and "*drug*". Indeed, there is concrete information in the use case scenarios of the health complaint system from which our approach recognizes the presence of this anonymous concept with the corresponding possible values. The important values of recall (91.67%) and completeness (90.91%) confirm that our approach generated a good number of pertinent domain models' elements, regardless of some exceptions caused by the full automation of our approach.

Finally, we note high precision, recall, F-measure, completeness and correctness for the RSL requirements models generated by our tool and which exceed 85% in all cases. These results reflect that our tool is able to build RSL requirements models of high quality, which are similar to the pertinent ones developed by the RSL expert. Consequently, we deduce that the ReDSeeDS inputs generated by our approach are of high quality and help developers in programming by saving their

Table 4
Evaluation of the efficiency the Java code generated by our approach.

Averages	Precision	Recall	F-measure
Packages	92.54%	94.33%	93.43%
Classes	89.07%	91.21%	90.13%
Attributes	87.92%	90.26%	89.07%
Methods	86.11%	54.03%	66.40%

time, and thus money for the companies. In this latter context, the next section deals with the assessment of our approach efficiency in terms of the generated Java code.

4.2. Evaluating the efficiency of the Java code generated by our approach

Goal: In this section, we decided to give an empirical study on the main motivation of our approach, i.e. producing good source code from input language-independent scenarios, regardless of the domain of the target software application. In other terms, this second evaluation deals with the assessment of the Java code generated by our approach, and thus our tool.

Subject and Preparation: To evaluate the efficiency of the Java code generated by CRec-tool, we used four applications developed within the company BRAIN PRO[4] [61]. The staff of this company allowed us to carry out our empirical study within their company and on their own products (developed in Java by following a model/View/Presenter pattern). More specifically, our study deals with the following four applications, which belong to strictly different domains:

- An Enterprise Resource Planning (ERP) application, including different modules, such as purchasing management, sales management, stock management, customer management, supply management, etc.
- A Medical management application, which includes the appointments management, the management and the follow-up of medical records, the staff management, etc.
- A Hotel management application, which handles the information about the hotel's rooms, services, excursions, booking, etc.
- A Campus management application that manages students information, their inscriptions, their scholarships, the teachers, etc.

A good pre-study and design of the above-mentioned applications were made by the company developers.

Task: The only restriction made by the company was to preserve the confidentiality of their source code. Indeed, they did not give us these code for copyrights reasons. However, they promised us to compare our code to theirs and to provide us with quantitative values as well as their general feedback. More specifically, they provided us with the use case diagrams and scenarios of the above applications. The scenarios, having quite complex and ambiguous sentences structures, were written in French. We used these scenarios as inputs to the CRec-tool and we got the corresponding Java code. Then, we gave the latter to the programmers responsible for the development of these applications. After a period of three weeks (for time constraints), these programmers gave us the TP, FP and FN approximated values in terms of the generated Java code' elements (i.e. packages, classes, attributes and methods). We employed these values to measure the precision, the recall and the F-measure in terms of packages, classes, attributes and methods. Table 4 shows the obtained averages values.

Evaluation: Table 4 shows important average values of precision, recall and F-measure in terms of packages, classes, attributes and methods, which exceed 87%. This reflects the ability of our approach to produce useful code elements. Besides, the programmers judged that the low number of false positives can be easily removed before completing the generated code.

Concerning the relatively low values of recall in terms of methods (ranging from 39% and 61%), the programmers, involved in our evaluation mentioned that the majority of the methods' signatures are correctly stated in the Java code generated by our tool, added to the constructors' code. This confirms the good average precision values in terms of methods (86.11%). Moreover, they mentioned that the effective code of many methods were absent in our source code. This is obvious while a use case scenario could not contain a detailed description of the treatment intra-operations, and thus intra-methods. This confirms the low values of recall in terms of methods.

It is worth mentioning that the programmers appreciated the source code generated by our approach. They noted that relying on our approach (and thus our tool) is rewarding by decreasing the development time and effort, thanks to the generated efficient Java code. For instance, they stated that the development of the ERP application took nearly one year and a half of working days, to be fulfilled and ready for sale. They also mentioned that this period of time would be reduced, nearly, to one year if they used our tool. Obviously, this would make the company gain twice: (i) by avoiding paying the salary of the programmers, involved within this application, for six months, and (ii) by earning other eventual markets to sell this product in the gained six months, and thus much money.

[4] This is an IT company, created in 2011. which proposes different business solutions, mobile applications, web solutions, etc.

Table 5
Evaluation of the ability of our approach to extract relevant XML code elements conforming to the input requirements.

Averages	pUCS	rUCS	FUCS	Relevance	Error
Objects	88.33%	96.66%	92.31%	91.02%	8.07%
Attributes	83.18%	92.13%	87.43%	81.12%	9.17%
Operations	87.41%	94.36%	90.75%	84.33%	7.06%
Instructions	82.12%	96%	88.52%	95.32%	8.91%

4.3. Evaluating the ability of our approach to extract relevant XML code elements conforming to the input requirements

Goal: The overall objective of our third evaluation is to show the ability of our approach, and thus CRec-tool, to extract the XML code elements, conform to the input language-independent requirements. Indeed, it treats the quality of the semantic model outputs and their usefulness for relevant information extraction from texts. This property is one of the key properties when evaluating a model transformation technique for code generation because it influences the heuristics used during the synthesis mechanism.

Subject and Preparation: To evaluate the conformity of the generated information from our approach application, in general, and the semantic model, in particular, on language-independent scenarios, we assigned the same use case scenarios, used for our first evaluation (see section 4.1), to a class containing 15 students in an XML course. This class holds good and well-ranked students who are familiar with the XML language and able to build reliable XML code. Besides, we gave them our proposed XML structure for the target XML code (see section 3.1).

Task: The students were divided into five groups, each of which includes three students with distributed skills. In fact, we appointed a leader class for each group in order to maintain a balanced structure of these groups and obtain better results in terms of the identified XML code elements. Besides, each group was given the use case scenarios belonging to one domain. The students of each group had three hours (i.e. two lab sessions) to write their XML code according to the given target structure. Evidently, these groups understood the language in which the given use case scenarios were written. Obviously, this evaluation is independent of the first evaluation and of the proposed approach towards getting ReDSeeDS inputs. In other terms, this evaluation focused only on the generated XML code, independently of the syntax of the target code generator. As we have mentioned before, the output implementation of the semantic model is a standard XML code which can be easily understood and manipulated by all techniques and tools.

We computed the values of true positives (TP), false positives (FP), false negatives (FN) and true negatives (TN, i.e., the number of none pertinent XML nodes which were not retrieved by our tool). These values are used to measure the precision, the recall and the F-measure (see equations (3), (4) and (5)) of our XML code in terms of use case scenarios (we call them respectively *pUCS*, *rUCS* and *FUCS*). We also calculated the Relevance and the Error measures [62] as follows:

$$Relevance = \frac{TP + FP}{TP + FP + TN + FN} \times 100 \tag{8}$$

$$Relevance = \frac{FP + TN}{TP + FP + TN + FN} \times 100 \tag{9}$$

Table 5 shows the results of the mentioned comparison.

Evaluation: As we have mentioned before, our work aims at maximizing simultaneously the precision and the recall rates, and consequently the F-measure. Table 5 shows clearly the obtained values of the F-measure, which are close to 100%, with an average of 89.75%. Similarly, we note important values of relevance, reaching even 97% in the case of the Emptio application, with an average that exceeds 81%. In contrast, the error rates are low, ranging between 7.06% and 9.17%. These facts reflect the ability of our method to generate XML code, having a significant compliance with the input use case scenarios of the corresponding systems. This confirms the efficiency of our semantic model in terms of extracting relevant information from language-independent input scenarios.

4.4. Threats to validity

A key issue when performing experiments in the validity of the results: is the study designed and performed in a sound and controlled manner? are the statistical tests used correctly? are the issues under investigation really investigated in the study? to which domain are the results possible to generalize? [63]. In the remainder of this section, we discuss the threats of the our approach based on four types of threats proposed by Wohlin et al. [54], namely *conclusion validity, internal validity, construct validity* and *external validity*.

Conclusion validity: Conclusion validity concerns the relation between the treatments and the outcome of the experiment [63]. In other terms, it is the degree to which conclusions we reach about relationships in our data are reasonable [64]. One conclusion threat against our approach is its dependence on the input requirements. More specifically, the more complete input use case scenarios are, the better results we get. In fact, the evaluation of our approach shows interesting findings in

terms of the adopted measurement values (i.e. precision, recall, F-measure, etc.) because we had good scenarios, detailed and written by experts. In this case, the programmer gains much time. In other terms, if the input use case descriptions are poor in terms of the contained steps, our approach generates a weak XML code, and thus RSL notions and RSL scenarios inducing a weak Java code which would necessitate a deep programmer intervention. This threat is partially due to the full automation of our approach. The second threat is the approximated values of TP, FP and FN given by the programmers in our second experimentation (see section 4.2). Although these programmers are objective, we had better to rather collect this data through a survey in order to be sure of the reliability of the given TP, FP and FN values.

Internal validity: Besides the above conclusion threat, we identify two internal threats to validity. Internal validity is the approximate truth about inferences regarding cause-effect or causal relationships [64]. A major concern is the manual cost resulting from studying and implementing the specification of each natural language within the corresponding NOOJ syntactic/semantic grammar in order to build the mapping rules of the semantic model patterns. This fact requires the creation of a syntactic/semantic grammar for each supported language, which necessitates a deep study particularly of the structures of its sentences and phrases in order to build a powerful grammar that generates high quality mapping rules. For instance, the subject precedes a verb in an English or a French declarative sentence. In contrast, the verb comes before the subject in an Arabic sentence. Similarly, for the declarative French sentence, the direct object complement (COD, "*complément d'objet direct*" in French) is followed by the indirect object complement (COI "*complément d'objet indirect*" in French). However, in German, when we want to make the stress on the COI, we can put it before the COD. The second internal threat stems from the inability of our approach to assemble the redundant instructions, having the same context, into a method which can be called if needed. For example, in the Cash withdrawal scenario, we note the presence of four steps in that scenario, which correspond to the same treatment (bank card confiscating) and depend only on the confiscating cause. These steps (and thus the corresponding involved treatment) were repeated in four alternative flows of this scenario. Therefore, our approach generated the four steps of the same RSL scenarios, four times within the RSL scenario of the use case "*Withdraw cash*". In contrast, the RSL expert created an intermediate RSL scenario "*Confiscate card*" in which he put the RSL sentences corresponding to the afore-mentioned steps. This intermediate scenario is invoked when needed. This fact reduces the number of RSL scenarios' sentences.

Construct validity: Construct validity refers to the degree to which inferences can legitimately be made from the operationalizations in your study to the theoretical constructs on which those operationalizations were based [64]. One major construct validity of our approach is its ability to treat use case scenarios written in any language which remains theoretic and hard to handle in practice. Indeed, a grammar should be practically created for each language to be treated by our approach in order to synthesize the mapping rules and translate them to English. The translation of the effective expression involved within the mapping rule does not constitute a problem because we used Google Translator which already handles many languages. Our main concern is to build the mapping rules using grammars. However, we believe that this is not an important problem while creating this grammar is done only one time. Then it becomes ready to treat the input instructions in that language.

External validity Threats to external validity are conditions that limit our ability to generalize the results of our experiment to real practice (e.g., in the industry) [65]. One external threat is the NOOJ environment, used for the implementation of the semantic model mapping rules. This environment is time consuming in parsing input texts and generating the corresponding mapping rules. For example, it took more than one minute to extract the mapping rules for the steps of the scenario "*Send message*" from the GoPhone system. Moreover, we have to dispose a NOOJ dictionary for each supported natural language to be treated by our approach. However, NOOJ has rich dictionaries written in many languages such as French, English, German, Arabic, Spanish, etc. Besides, new dictionaries for new languages are integrated within NOOJ each year.

5. Conclusion

This paper presented a new approach as a first step towards naturalistic programming. It provides for writing language-independent requirements (represented as multi-lingual use case scenarios) from which the target code is automatically generated.

To do so, our approach operates in three stages. First, it transforms the use case scenarios into the semantic model patterns by creating the corresponding mapping rules. The semantic model allows our approach to use semantic information when exploring and interpreting the syntax and semantics of the requirements, independently of the language in which they are written. Secondly, our approach extracts the implementation of the generated mapping rules as an XML code by following a specific structure. Thanks to the standardized structure of XML, the resulting XML code can be used for different purposes; as illustrated in this paper, this XML code can be used to build the inputs accepted by the code generator ReDSeeDS. For this code generator, our approach can transform the generated XML code into RSL requirements model and RSL domain model, the two mandatory inputs of ReDSeeDS which transforms them to a Java code, following a Model/View/Presenter architecture.

Unlike existing approaches, ours is very simple because it does not necessitate any pre-study of a particular language to be used. It focuses on handling the hard case where the scenarios are written in different natural languages, thanks to

the semantic model. Moreover, our approach has the potential to extract adequate information from ambiguous language-independent use case scenarios, leading to quite complete and correct ReDSeeDS inputs, consequently producing good Java code.

Our approach was implemented within the CRec-tool which automates all its steps. Using CRec-tool, our approach was quantitatively evaluated on five systems by comparing the precision, the recall, F-measure, completeness and correctness rates of the RSL models generated by our tool versus other ones built by RSL experts. These measurements showed that our approach has a high capability in identifying almost the majority of the true RSL models, i.e., those proposed by the experts involved in this evaluation. Indeed, our approach demonstrated an average precision (respectively recall) of 80.49% and 89% (respectively 92.4% and 91%) in terms of RSL domain models and RSL requirements models, respectively. Furthermore, we got an average completeness in terms of RSL domain model (respectively requirements model) which is equal to 91.74% (respectively 94.5%), and an average correctness of 92.55% (respectively 92%) in terms of RSL domain model (respectively requirements model). Besides, the students' feedback showed that the XML code generated by our approach are conform to the input language-independent use case scenarios. In fact, we obtained good average values of F-measure (between 87.43% and 92.31%) and relevance (between 81.12% and 95.32%) in terms of XML nodes "*object*", "*attribute*", "*operation*" and "*instruction*", versus low error averages (between 7.06% and 9.17%). Furthermore, we assessed the efficiency of the Java code produced by our tool on four software applications based on programmers feedback. We got good results of precision, recall and F-measure (varying between 66.4% and 93.43%). Besides, these programmers appreciated the outputs of our tool and found them efficient by decreasing the programming time.

Our approach is useful for companies that start the development process by collecting information related to the new application domain. This information consists in the use case scenarios involved within the design of these applications (e.g., collected from available documents or their own developed scenarios). Employing our tool, developers could save their time by having access to quite complete and correct source code elements. Furthermore, because our approach works on several natural languages, the development task would be easier within large companies that have employees speaking several languages. In this case, the employees could use scenarios written in their mother tongues, and get the target code whose elements names (e.g., classes, attributes, operations and parameters names) are expressed in the English language. The resulting code would be understood and easily manipulated by different employees working on the same application. Finally, our approach focuses in the most common area of programming, which is the human computer interaction (HCI) programming based on GUI, thanks to the generated code following the MVP architectural pattern.

We believe that naturalistic programming will have a prominent place in the future of programming languages. To support this, in our future works, we will conduct an evaluation on a larger set of use cases to confirm the presented results. Another practical extension of the herein presented work is the integration of other languages within CRec-tool in order to prove its multi-lingual aspect on a larger scale. Furthermore, we will integrate other modules within our tool to build the syntax of other existing code generators from the generated XML code. Finally, we plan to build an API to establish the communication between the RSL models generated by our approach and the ReDSeeDS code generator. Indeed, this generator is based on a personalized version of the IDE Eclipse, with an adapted tree structure for the RSL models. Besides, it only accepts its specific formats. Consequently, we should contact the ReDSeeDS developers in order to have more information on these formats. Then, we will develop an API, which will accept, as inputs, the URLs of the RSL models files generated by CRec-tool. Then, this API will parse these files' content to the required formats. Finally, it will communicate with ReDSeeDS via web services or via requests, in order to get the target code. The latter process should also be implemented within this ReDSeeDS.

Appendix A. CRec-tool

To have a practical use of the herein proposed approach, we decided to implement the involved process within a Java tool called CRec-tool which supports the French and the English languages. In the remainder of this appendix, we explain the different functionalities integrated within our tool.

A.1. F1: Building mapping rules

This functionality enables generating the mapping rules for the input scenarios within TXT files by following the patterns described in Section 3.1. This representation as mapping rules will be then used by CRec-tool to build the corresponding XML code. To this end, our tool does not follow a specific algorithm for the synthesis task; in contrast, it relies on grammars. In this sight, CRec-tool is based on the linguistic development environment, NOOJ [66]; this latter includes many dictionaries, morphological and syntactic grammars, as well as tools to create, check, debug, adapt, maintain, and share them. Dictionaries and grammars are applied on input texts in order to locate morphological, syntactic and lexical tags. The results of applying dictionaries are annotations. Besides, using grammars may generate a concordance table where the recognized expressions are in the middle column. The resulting concordance can be exported as a TXT file.

The extraction of the different mapping rules that match each sentence to its formal representation according to the semantic model patterns involves three important steps, namely the extraction of the *lexical knowledge*, the *semantic knowledge* and the *grammatical knowledge*.

Table 6
The content of some sub-graphs that belong to our syntactic/semantic grammars.

Node	Structure
:Noun_Group	[:Quantity_Ordinality] [:Adjectival_Group] <N>* + :References + [:Quantity_Ordinality] [:Adjectival_Group] "<WF>*" + :Compression_Noun
: Explicit_Reference	<DET> :Noun_Group + <DET> :Adjectival_Group
: Symbol_self_reference	[<DET>] :Noun_Group "<WF>*"
:Possession_Concept _Relation_Statement	:Noun_Group <POSS> [<PREP>]:Noun_Group
:Sentence	<LINK+TYP=condition> :Statement + <LINK+TYP=contrary> :Statement + <LINK+TYP=Adversative> <WF>* + <LINK+TYP=Concessive> <WF>* + <LINK+TYP=Explicative> <WF>* + <LINK+TYP=Restrictive> <WF>* + <LINK+TYP=Final> <WF>* + <LINK+TYP=Temporal> <WF>* + <LINK+TYP=Location> <WF>* + :Loop

A.1.1. Lexical knowledge extraction

The lexical knowledge enables the distinction between the different words that exist in a sentence. These words are included in a NOOJ electronic dictionary. An electronic dictionary contains sufficient information that is attached to lexical entries and usable for various stages of analysis. A word is represented in a NOOJ electronic dictionary by one of these two forms: (i) *Inflected form:* it is the concrete form in which the word occurs in the sentence; (ii) *Basic form (lemma):* it is the root of the word, as a unit in the dictionary.

NOOJ contains several dictionaries and morphological grammars for different languages. These dictionaries and grammars identify a fixed set of word forms and associate them with some linguistic information. In this context, we relied on the already existing NOOJ dictionaries and morphological grammars to extract the mapping rules. In addition, we created other dictionaries (for each accepted language, namely English and French) which contain some specific entries related to the semantic model features. For instance, they contain different links, compression phrases and quantities, associated with their types. In fact, we were obliged to create additional dictionaries because the existing ones are in their compiled forms; in other words we did not have the possibility to modify their contents and adapt them to our needs.

A.1.2. Semantic knowledge extraction

The semantic knowledge focuses particularly on the predicates and the links. In fact, if a word/expression is a link, it has to belong to a semantic class ("final", "explicative", "condition"...). Similarly, if a word is a predicate, it has to belong to a predicate class already defined (action or state). Each predicate has a number of obligatory arguments. Each argument has a semantic role. For example, the predicate "*Send*" belongs to the class "*action*" and its obligatory semantic roles arguments are "*agent+object+destination*". Besides, the role "*agent*" is not obligatory in case of passive forms.

A.1.3. Grammatical knowledge extraction

The grammatical knowledge deals with how sentences are built structurally. In this step, we tried to extract the structure of an input requirement according to the semantic model, based on grammatical rules developed with NOOJ. For this reason, we created a syntactic/semantic grammar for each language (English and French), each of which contains about 80 graphs that treat the input requirements and generates the corresponding mapping rules in English. Practically speaking, we have to build a syntactic/semantic grammar for each natural language to be treated by our tool in order to generate the mapping rules of an input text written in that language. More specifically, taking into account a text written in a language, L, different from English, our tool starts by exploiting this text by using the corresponding developed grammar, as well as the required NOOJ dictionaries appropriate to L. Then, our tool generates the corresponding mapping rules containing terms belonging to the language L. To solve the problem of the mapping rules translation to the English language, CRec-tool uses the *Google Translate API* [67] in order to transform the components of the generated mapping rules (i.e. the names of concepts, properties, states, actions, etc.) into English.

Table 6 shows the content of some sub-graphs that belong to our syntactic/semantic grammars, where: (i) The character ":" denotes a sub-graph; (ii) A simple lexical entry is put between "<>". The asterisk is used to represent a recursive path of an element (lexical entry, sub-graph...); (iii) The "+" denotes a new path which can be taken; (iv) The "<WF>" is a NOOJ element which represent any word form; (v) <DET> is a NOOJ element which denotes a determinant, such as "the" in English, "le" in French...

To summarize, CRec-tool performs the creation of a mapping rule corresponding to a an input requirement by three steps:

1. Selecting the dictionary of a language (the language in which the input requirements are written) and applying the syntactic analysis ⇒ *Result:* Annotated text.
2. Selecting and applying the syntactic/semantic grammar according to the language of the requirements ⇒ *Result:* Mapping rules in the used language.
3. Translating the mapping rules into English ⇒ *Result:* Mapping rules in English.

CRec-tool stores the generated mapping rules of an input use case scenario (UCS) within a file "$UCS.txt$" whose name corresponds to the scenario's name.

A.2. F2: Building XML code

This functionality enables building the XML code that corresponds to the generated mapping rules within the file "*UCS.txt*". This XML code is conform to the implementation of the semantic model patterns. Our tool stores this code in the file "*UCS.xml*". Moreover, CRec-tool creates a JTree for each XML code. More specifically, we created some methods to access, locate, create and add nodes to the parent ones. We also produced the whole XML tree. For example, the following method enables CRec-tool to create XML nodes "<*instruction*>" from the file "*UCD.txt*":

```
public void constructInstructionNode(String UCD){
  [...] /* Variables initialization here */
  [...] /* Creating the XML file which will contain the XML nodes <instruction> */
  [...] /* Opening \xch{the}{the the} file "UCD" which contains the mapping rules */
  while ((MR=br.readLine())!=null){
    if(MR.indexOf("statement,")>-1){
      separated=MR.split("\\(");
      for(int i=0;i<separated.length;i++){
        if(separated[i].indexOf("action,")>-1){
          //To extract the action predicate from the MR
          begin=separated[i].indexOf(",")+1;
          end=separated[i].indexOf(")",begin);
          predicate=separated[i].substring(begin,end);
        }else{
          if(separated[i].indexOf("agent,")>-1){
            //To extract the agent from the MR
            begin=separated[i].indexOf(",")+1;
            end=separated[i].indexOf(")",begin);
            agent=separated[i].substring(begin,separated[i].length());
            agent="("+agent.substring(0,agent.length()-3)+")";
          }else{
            if(separated[i].indexOf("object,")>-1){
              //To extract the object from the MR
              begin=separated[i].indexOf(",")+1;
              end=separated[i].indexOf(")",begin);
              arguments.add("("+separated[i].substring(begin,end)+")");
            }
        [...] //Extracting the other eventual arguments
      // Inserting XML <instuction> nodes within the XML file
      writer.print("<instruction type=action>"+agent+" "+predicate+" ");
      Iterator<String> it = arguments.iterator();
      while (it.hasNext()) {
          String n = it.next();
          writer.print(n+" ");
          System.out.print(n+" ");
      }
      writer.println("</instruction>");
      arguments.clear();
      }
  [...] /* Closing the files */
}
```

A.3. F3: Redundancy elimination from XML code

This functionality enables calculating the matrix of cosine between documents and queries. It takes as inputs a list of clauses (i.e., the XML nodes names) and constitutes the corresponding queries and documents. In particular, the construction of documents is performed by interfacing with the WordNet ontology to get the synonyms of words. As a result, CRec-tool generates the matrix of cosine. After that, this tool regroups the names into semantic classes according to the obtained cosine matrix. Thus, our tool eliminates the redundancy among the XML nodes according to the obtained semantic classes.

A.4. F4: Building the RSL models required by ReDSeeDS

This functionality allows building the RSL domain model and requirements model from the generated XML code. In this context, Algorithm 1 and 2 are implemented within CRec-tool, which stores the resulting RSL models into XML files. These files are used as inputs for the ReDSeeDS generator.

References

[1] R. Knöll, M. Mezini, Pegasus: first steps toward a naturalistic programming language, in: Companion to the 21st ACM SIGPLAN Symposium on Object-oriented Programming Systems, Languages, and Applications, ACM, New York, NY, USA, 2006, pp. 542–559.
[2] R. Knöll, V. Gasiunas, M. Mezini, Naturalistic types, in: Proceedings of the 10th SIGPLAN Symposium on New Ideas, New Paradigms, and Reflections on Programming and Software, ACM, New York, NY, USA, 2011, pp. 33–48.
[3] L. Khaylov, Implementation of the Naturalistic Programming Language Pegasus_f, Master's Thesis, Darmstadt University of Technology, Germany, 2009.
[4] I. Jacobson, Object Oriented Software Engineering: A Use Case Driven Approach, Addison–Wesley, USA, 1992.
[5] A. Cockburn, Writing Effective Use Cases, 1st edition, Addison–Wesley Longman Publishing Co., Inc., Boston, MA, USA, 2000.
[6] L.L. Constantine, L.A.D. Lockwood, Structure and style in use cases for user interface design, in: Object Modeling and User Interface Design, Addison–Wesley Longman Publishing Co., Inc., Boston, MA, USA, 2001, pp. 245–279.
[7] K. Bittner, Use Case Modeling, Addison–Wesley Longman Publishing Co., Inc., Boston, MA, USA, 2002.
[8] W. Nowakowski, M. Smialek, A. Ambroziewicz, T. Straszak, Requirements-level language and tools for capturing software system essence, Comput. Sci. Inf. Syst. 10 (4) (2013) 1499–1524, https://doi.org/10.2298/CSIS121210062N, ComSIS Consortium.
[9] M. Mefteh, N. Bouassida, H. Ben-Abdallah, Feature model extraction from documented UML use case diagrams, Ada User J. 35 (2) (2014) 108–117, Ada Language UK Ltd.
[10] M. Mefteh, N. Bouassida, H. Ben-Abdallah, Mining feature models from functional requirements, Comput. J. 59 (7) (2016) 1–21, https://doi.org/10.1093/comjnl/bxw027, Oxford University Press.
[11] M. Mefteh, N. Bouassida, H. Ben-Abdallah, Implementation and evaluation of an approach for extracting feature models from documented UML use case diagrams, in: Proceedings of the Annual ACM Symposium on Applied Computing, SAC '15, ACM, New York, NY, USA, 2015, pp. 1602–1609, http://doi.acm.org/10.1145/2695664.2695907.
[12] H. Liu, H. Lieberman, Metafor visualizing stories as code, in: Proceedings of the 10th International Conference on Intelligent User Interfaces, ACM, New York, NY, USA, 2005, pp. 305–307.
[13] A. Cozzie, S. King Macho, Writing Programs with Natural Language and Examples, Tech. Rep., University of Illinois at Urbana-Champaign, 2012.
[14] T. Gvero, V. Kuncak, Synthesizing Java expressions from free-form queries, in: Proceedings of the 2015 ACM SIGPLAN International Conference on Object-Oriented Programming, Systems, Languages, and Applications, ACM, New York, NY, USA, 2015, pp. 416–432.
[15] B. Bousetta, O. El Beggar, T. Gadi, A methodology for CIM modelling and its transformation to PIM, J. Inf. Eng. Appl. 4 (2) (2013) 1–21, Academic Hosting & Event Management Solutions.
[16] M. Kardos, M. Drozdovà, Analytical method of CIM to PIM transformation in model driven architecture (MDA), J. Inf. Organ. Sci. 34 (1) (2010) 89–99, Faculty of Organization and Informatics, University of Zagreb, Croatia.
[17] J. Franců, P. Hnětynka, Automated Generation of Implementation from Textual System Requirements, Springer, Berlin, Heidelberg, 2011, pp. 34–47.
[18] M. Smialek, W. Nowakowski, From Requirements to Java in a Snap, Springer International Publishing, Switzerland, 2015.
[19] M. Potel, MVP: Model–View–Presenter: The Taligent Programming Model for C++ and Java, Tech. Rep., Taligent, Inc., 1996.
[20] D. Muthig, I. John, M. Anastasopoulos, T. Forster, J. Dörr, K. Schmid, GoPhone – A Software Product Line in the Mobile Phone Domain, Tech. Rep., No. 025.04/E, Version 1.0, Fraunhofer IESE, 2004.
[21] G. Glavas, J. Snajder, Construction and evaluation of event graphs, Nat. Lang. Eng. 21 (4) (2015) 607–652, https://doi.org/10.1017/S1351324914000060, Cambridge University Press, United Kingdom.
[22] C.J. Fillmore, Frame semantics and the nature of language, in: S. Harnad (Ed.), Origins and Evolution of Language and Speech, Anu. N.Y. Sci. (1976) 155–202, https://doi.org/10.1111/j.1749-6632.1976.tb25467.x.
[23] J. Ruppenhofer, M. Ellsworth, M.R.L. Petruck, C.R. Johnson, J. Scheffczyk, FrameNet II: extended theory and practice, http://framenet.icsi.berkeley.edu, 2010.
[24] C.F. Baker, C.J. Fillmore, J.B. Lowe, The Berkeley FrameNet project, in: Proceedings of the 36th Annual Meeting of the Association for Computational Linguistics and 17th International Conference on Computational Linguistics – Volume 1, ACL '98, Stroudsburg, PA, USA, Association for Computational Linguistics, 1998, pp. 86–90.
[25] Marie-Claude L'Homme, Terminologie de l'environnement et sémantique des cadres, in: Congrès Mondial de Linguistique Française, SHS Web of Conferences, vol. 27, 2016, pp. 1–14.
[26] A. Fatolahi, S.S. Somé, T.C. Lethbridge, A model-driven approach for the semi-automated generation of web-based applications from requirements, in: Proceedings of the Twentieth International Conference on Software Engineering & Knowledge Engineering, SEKE'2008, San Francisco, CA, USA, July 1–3, Knowledge Systems Institute Graduate School, 2008, pp. 619–624.
[27] T. Yue, Automatically Deriving a UML Analysis Model From a Use Case Model, Ph.D. Thesis, Ottawa, Canada, 2010.
[28] Y. Rhazali, Y. Hadi, A. Mouloudi, CIM to PIM transformation in MDA: from service-oriented business models to web-based design models, Int. J. Softw. Eng. Appl. 10 (4) (2016) 125–142, https://doi.org/10.14257/ijseia.2016.10.4.13, Science & Engineering Research Support Society.
[29] M. Śmialek, N. Jarzebowski, W. Nowakowski, Runtime semantics of use case stories, in: IEEE Symposium on Visual Languages and Human-Centric Computing, VL/HCC, IEEE Computer Society, 2012, pp. 159–162.
[30] A. Kalnins, S.T.M. Smialek, E. Kalnina, E. Celms, W. Nowakowski, Developing software with domain-driven model reuse, in: Handbook of Research on Innovations in Systems and Software Engineering, IGI Global, 2014.
[31] M. Pathan, A. Patkar, S. Surve, Automatic partial code generation using class and sequence diagrams, Int. J. Recent Innov. Trends Comput. Commun. 4 (3) (2016) 365–368, Auricle Technologies Pvt. Ltd., Bikaner.
[32] O.E. Beggar, B. Bousetta, T. Gadi, Automatic code generation by model transformation from sequence diagram of system's internal behavior, Int. J. Comput. Inf. Technol. 1 (2) (2012) 129–146.
[33] D. Price, E. Rilofff, J. Zachary, B. Harvey, NaturalJava: a natural language interface for programming in Java, in: Proceedings of the 5th International Conference on Intelligent User Interfaces, ACM, New York, NY, USA, 2000, pp. 207–211.
[34] E. Özcan, S.E. Seker, Z.I. Karadeniz, Generating Java class skeleton using a natural language interface, in: Natural Language Understanding and Cognitive Science, Walter de Gruyter GmbH, Porto, Portugal, 2004, pp. 126–134.
[35] K. Czarnecki, Generative Programming – Principles and Techniques of Software Engineering Based on Automated Configuration and Fragment-Based Component Models, Ph.D. Thesis, Ilmenau Technical University, Germany, 1999.
[36] P. Kruchten, The Rational Unified Process: An Introduction, Addison–Wesley Longman Publishing Co., Inc., USA, 1999.
[37] R. Wirfs-Brock, Designing Scenarios: Making the Case for a Use Case Framework, Tech. Rep., Smalltalk Report, 1993.
[38] M. Ali, H.B. Abdallah, F. Gargouri, Towards a validation approach of up conceptual models, in: European Conference on ModelDriven Architecture, Foundations and Applications Workshop on Consistency in Model DrivenEngineering, Germany, 2005, pp. 143–154.
[39] C. Alexander, S. Ishikawa, M. Silverstein, M. Jacobson, I. Fiksdahl-King, S. Angel, A Pattern Language, Oxford University, USA, 1977.
[40] G. Sunyé, A. Le Guennec, J.-M. Jézéquel, Design Patterns Application in UML, Springer, Berlin, Heidelberg, 2000, pp. 44–62.
[41] E. Gamma, R. Helm, R. Johnson, J. Vlissides, Design Patterns: Elements of Reusable Object-Oriented Software, Addison–Wesley Longman Publishing Co., Inc., Boston, MA, USA, 1995.

[42] M. Hafiz, P. Adamczyk, R.E. Johnson, Organizing security patterns, IEEE Softw. 24 (4) (2007) 52–60, https://doi.org/10.1109/MS.2007.114, IEEE Computer Society.
[43] M. Hafiz, P. Adamczyk, R.E. Johnson, Growing a pattern language (for security), in: Proceedings of the ACM International Symposium on New Ideas, New Paradigms, and Reflections on Programming and Software, Onward! 2012, ACM, New York, NY, USA, 2012, pp. 139–158.
[44] S. Brahe, B. Bordbar, A pattern-based approach to business process modeling and implementation in web services, in: D. Georgakopoulos, N. Ritter, B. Benatallah, C. Zirpins, G. Feuerlicht, M. Schoenherr, H.R. Motahari-Nezhad (Eds.), Service-Oriented Computing ICSOC 2006: 4th International Conference, Chicago, IL, USA, December 4–7, 2006, Springer, Berlin, Heidelberg, 2007, pp. 166–177.
[45] O. Barros, Business process patterns and frameworks: reusing knowledge in process innovation, Bus. Process Manag. J. 13 (1) (2007) 47–69, https://doi.org/10.1108/14637150710721122, Emerald Publishing Limited.
[46] H. Ben-Abdallah, J. Feki, M. Ben Abdallah, A multidimensional pattern based approach for the design of data marts, in: Progressive Methods in Data Warehousing and Business Intelligence: Concepts and Competitive Analytics, IGI Global, 2009, pp. 172–192.
[47] R.S. Scowen, Extended BNF – a generic base standard, in: Proceedings of the 1993 Software Engineering Standards Symposium, SESS'93, 1993, http://www.cl.cam.ac.uk/~mgk25/iso-14977-paper.pdf.
[48] W. Jirapanthong, A. Zisman, XTraQue: traceability for product line systems, Softw. Syst. Model. 8 (1) (2009) 117–144, https://doi.org/10.1007/s10270-007-0066-8, Springer, Berlin, Heidelberg.
[49] E. Tourabi, R. Athousaki, Lexical Semantics of Verbs and Thematic Roles, Course Material for Lexical Semantics, Software Engineering Institute, 2004.
[50] M. Mefteh, N. Bouassida, H. Ben-Abdallah, The semantic model, https://sites.google.com/site/thesemanticmodel/, 2018.
[51] C.D. Manning, P. Raghavan, H. Schütze, Introduction to Information Retrieval, Cambridge University Press, New York, NY, USA, 2008.
[52] G.A. Miller, WordNet: a lexical database for English, Commun. ACM 38 (1) (1995) 39–41, https://doi.org/10.1145/219717.219748, New York, NY, USA https://wordnet.princeton.edu/.
[53] L. Skrzypek, P. Labeki, M. Śmialek, ReDSeeDS user Guide, Tech. Rep., Warsaw University of Technology, 2014.
[54] C. Wohlin, M. Höst, K. Henningsson, Empirical Research Methods in Web and Software Engineering, Springer, Berlin, Heidelberg, 2006, pp. 409–430.
[55] L.P. Tizzei, C.M.F. Rubira, J. Lee, A. Garcia, M. Barros, Health complaint system, http://www.ic.unicamp.br/~tizzei/phc/jss2013/, 2013.
[56] K. Bittner, I. Spence, Use Case Modeling, Pearson Education Inc., 2002, pp. 301–330.
[57] J. Kienzle, N. Guelfi, S. Mustafiz, Crisis management systems: a case study for aspect-oriented modeling, in: Transactions on Aspect-Oriented Software Development, VII: A Common Case Study for Aspect-Oriented Modeling, Springer, Berlin, Heidelberg, 2010, pp. 1–22, Ch. Crisis Management Systems: A Case Study for Aspect-Oriented Modeling.
[58] M. Smialek, N. Jarzebowski, W. Nowakowski, Translation of use case scenarios to Java code, Comput. Sci. 13 (4) (2012) 35–52, https://journals.agh.edu.pl/csci/article/view/44, Elsevier North-Holland, Inc., Amsterdam, The Netherlands.
[59] Elghadraoui, Rapport de gestion scolarité, Tech. Rep., ISTA NTIC SYBA Marrakech, 2013.
[60] C.R. van der Burg, T. Kirke, A. Rokic, Emptio – A Mobile Phone Application for Selfservice, Master's Thesis, Aalborg University, Germany, 2011.
[61] A. Chakroun, BRAIN PRO, http://brain-pro.com/index.php, 2011.
[62] D. Nakache, E. Metais, Evaluation: nouvelle approche avec juges, in: INFORSID'05 XXIII e congrès, Grenoble, X, France, 2005, pp. 555–570, https://hal.archives-ouvertes.fr/hal-01125014.
[63] T. Thelin, P. Runeson, C. Wohlin, T. Olsson, C. Andersson, Evaluation of usage-based reading – conclusions after three experiments, Empir. Softw. Eng. 9 (1–2) (2004) 77–110, https://doi.org/10.1023/B:EMSE.0000013515.86806.d4, Kluwer Academic Publishers.
[64] W.M. Trochim, Research methods knowledge base, http://www.socialresearchmethods.net/kb/, 2006.
[65] J.-M. Davril, E. Delfosse, N. Hariri, M. Acher, J. Cleland-Huang, P. Heymans, Feature model extraction from large collections of informal product descriptions, in: Proceedings of the 2013 9th Joint Meeting on Foundations of Software Engineering, ESEC/FSE 2013, ACM, New York, NY, USA, 2013, pp. 290–300.
[66] M. Silberztein, NOOJ: a linguistic development environment, http://www.nooj-association.org/index.php?option=com_k2&view=item&layout=item&id=4&Itemid=615, June 2015.
[67] T. U. R. P. for Google Translate, Google Translate API, 2011, https://www.programmableweb.com/api/google-translate.